**University of Padua**

Department of Mathematics "Tullio Levi-Civita"

Master Degree in Computer Science

# GeoMedia: A Social Media for Location-Based and Time-Sensitive Content Sharing

*Master's Thesis*

*Supervisor*

Prof. Claudio Enrico Palazzi

*Co-supervisor*

Dott. Lorenzo Perinello

*Candidate*

Alberto Morini

*Student ID*

2107783

---

Academic Year 2025-2026

Artificial Intelligence tool, specifically, ChatGPT (OpenAI), has only been used as spell checking and semantic error checking during the development of this thesis. No content, ideas, analysis, results or other material was generated by AI or LLM tools as a substitute for my own work.

*"You wanna know how I did it?*
*This is how I did it Anton.*
*I never saved anything for the swim back."*

— *Gattaca (1997)*

# Abstract

Over the past few decades, the use of mobile devices has increased drastically, almost replacing desktop software and web browsers. This shift has been mainly driven by the widespread adoption of smartphones not just for phone capabilities but also for the availability of integrated sensors such as GPS, gyroscopes and Bluetooth, whose, together allow richer and more context-aware user experiences.
Consequently, by leveraging these technologies, Online Social Network (OSN) platforms have been able to provide more advanced and sophisticated features, collecting larger amounts of metadata, interactions and daily engagement.

This thesis presents "GeoMedia", an open-source framework designed for sharing multimedia content, such as text, images, audio and files, anchored to real-world geographic locations.
By leveraging GPS data available on mobile devices, users can explore and publish digital content through an interactive map rendered by the application, creating a seamless integration between the digital and physical worlds. Moreover, the framework offers several features and covers various use cases, including restricting content visibility within a customizable distance range, enabling time-sensitive content or applying user-based restrictions for content creation and visualization. It also allows users to create sequential posts, forming an ordered chain for discovering posts and their related places.

Furthermore, this thesis analyzes the motivations behind GeoMedia, its critical aspects and design considerations, describes its system architecture and implementation and compares different technological paradigms. Finally, it explores potential applications and outlines possible future developments of the framework.

A previous version of this application was presented at the GoodIT 2025: International Conference on Information Technology for Social Good, held from September 3 to 5, 2025, in Antwerp, Belgium [1].

# Contents

# List of Figures

# List of Tables

# List of Codes

# Chapter 1

# Introduction

## 1.1 Overview

Technology has always influenced society, from the first introduction of the Personal Computer to the Internet, changing the world year after year; from personal life to workspaces, people are surrounded by software on their computers and mobile devices. Over the last decades, the investments in the development of technological innovations, both in hardware and software, as increased exponentially, shaping economical and social impact by integrating the latest features into everyday life. Smartphones and tablets have been the core engine that completely changed daily habits and digital usage; their wide adoption has reached almost three quarters of global population [59, 53], making users abandon traditional computers, especially for social media [52] and instant messaging platform [55], this trend bring a change on the design and development of applications into a 'mobile first' logic.

Compared to computers, mobile devices offer distinct functionalities and strengths that have facilitated their widespread adoption. These can be categorized according to the fundamental logic underlying any technology:

1. **Hardware**: the increase in computational performance[8], battery life[4], fast connectivity, and the integration of several built-in sensors[41] such as gyroscopes, GPS, NFC, light sensors, cameras, and vibration motors. These enable applications to interact with different fields such as health[16], education[44], and, through motion-sensors or cameras, an interaction with physical world, creating a more immersive user experience.

2. **Software**: the development and presence of mobile applications has played a crucial role. In fact, platforms such as Windows Phone were eventually abandoned and

discontinued due to the lack of applications [17, 60] compared to operating systems like 'iOS'[23] and 'Android'[27].

In this way, smartphones apps have became a key integration with daily life and physical world, for example, when planning a trip or just driving, smartphones, integrated with cars assists users via app like Google Maps[29] or Waze[21], providing not just directions but also real-time traffic updates, read events or information about available parking. This makes the surrounding environment enriched with digital details. Another example is 'Google Lens'[28], which can identify objects from a photograph, or Shazam[49] that through devices' microphone can recognize a playing song. Even in workplaces or public areas, technology integrations can be present, from the restaurant that provide a digital menu by a tablet, to customers checking the nutrition macros into a grocery store by scanning the barcode on the pack, to museums by scanning a QR Code enabling audio-guided tours or unlock Augmented Reality functionalities.

## 1.2 Online Social networks

Alongside the widespread adoption of smartphones and mobile devices, online social networks, have emerged as one of the most influential and important sectors of the technology market. Today, social media platforms rank among the firsts most used applications worldwide [46, 22], while the companies that provide them become some of the most bigger actors in global financial markets [47, 38, 34].

Online Social Networks (OSN) are the digitalized form of a Social Network (typically represented as a graph of people where links correspond to relationships) through the Internet[11]. For instance, Facebook, one of the largest platforms, represents a graph of friendships, where nodes (users) are connected via undirected edges, assuming that friendships are mutual. There are also different types of OSN, such as Instagram or Twitter, where the links between users are directed, implementing a following model.

These services provide a variety of functionalities, first of all instant messaging capabilities, both one-to-one and one-to-many, and, in addition, the ability to share a wide range of multimedia content, including images, videos, audio recordings, and other types of documents.This facility and advantages compared to other communication forms, like SMS, has led to their integration across diverse social, professional, and public contexts. The particularity of sharing multimedia content, commonly referred to as a post, has coined the term Social Media, where the published content represent the node of interaction between

users through comments or reaction (e.g., 'like' or 'unlike'), thus to replicate human behavior in dialogues and social interactions.

Despite the significant benefits offered by social media services, including enhanced connectivity, community creation, and information sharing, their adoption has also introduced various social and ethical challenges. Concerns regarding user privacy are particularly relevant, as it is often not respected by the service providers themselves, who collect personal information and interests in order to monetize them or enable surveillance practices[39]. Other serious issues include the dissemination of misinformation and fake news, nowadays further amplified by Artificial Intelligence-generated content, commonly known as 'deepfakes' [40]. Such content can influence public opinion and social behavior, not always in a positive way [58, 9].

Online Social Networks have also intensified common societal problems through phenomena such as 'echo chambers'[2], where users are repeatedly exposed to information and content that aligns with their existing beliefs and preferences. This continuous reinforcement can strengthen pre-existing beliefs, limit exposure to alternative perspectives, and, in some cases, contribute to ideological radicalization, including the adoption of hateful or extremist political views.

To address these issues, social media platforms have adopted various moderation and safety mechanisms, including automated content analysis systems, community guidelines and reporting tools. While these measures contribute reducing the spread of harmful content, they are not always fully effective, due to the scale and complexity of emerging innovations, such as 'deepfake' [15]. Consequently user reporting and human moderation aside of automated tools, continue to play a crucial role in identifying, evaluating, and removing content that violates policies or legal regulations[5].

# Chapter 2

# State of Art

GeoMedia positions itself among existing online social media platforms, by offering functionality centered on the sharing of geolocated content linked to specific latitude and longitude coordinates. Content is visible only within a configurable distance radius and can be more restricted with time-sensitive parameters or user-based policy. Thus to create a wide range of real-world applications, including scavenger hunts, interactive city guides, and any other location-based experiences.

The platform enables users to discover and interact with the physical environment through a map enriched by content catagorized into different topics collections, contributed by other users, who can upload pictures or other multimedia resources associated with specific locations.

Although several other social media platforms utilize the GPS capabilities of mobile devices to enhance user experience and provide location-aware features, none fully realize the underlying concept of GeoMedia or offer the same degree of flexibility for creating diverse application scenarios and exploration of uploaded content.

## 2.1 Foursquare Swarm

Foursquare Swarm is the spin-off of a formerly app, "Foursquare City Guide", which provide personalized recommendations of nearby places to the users' current location based on its interests or previous browsing history and check-ins (locations previously visited by the user).

Foursquare was discontinued in 2024 and its functionalities have been integrated into Swarm, which in addition allows users to share their position with relatives, maintaining a

personal lifelog of visited places, which can be shared with others.

For each visited location, users may leave reviews or upload multimedia content; however, these contributions are strictly tied to physical place, such as stores, cafés, restaurants or other kind of attractions as shown in Figure 2.1. Users cannot publish content at arbitrary geographic coordinates, nor can restrict post visibility, as all content is publicly accessible. The application further encourages the usage through a gamification system, in which users earn badges for visiting locations and uploading content. These mechanisms are broadly comparable to those implemented in other platforms such as Google Maps (see Section 2.4).

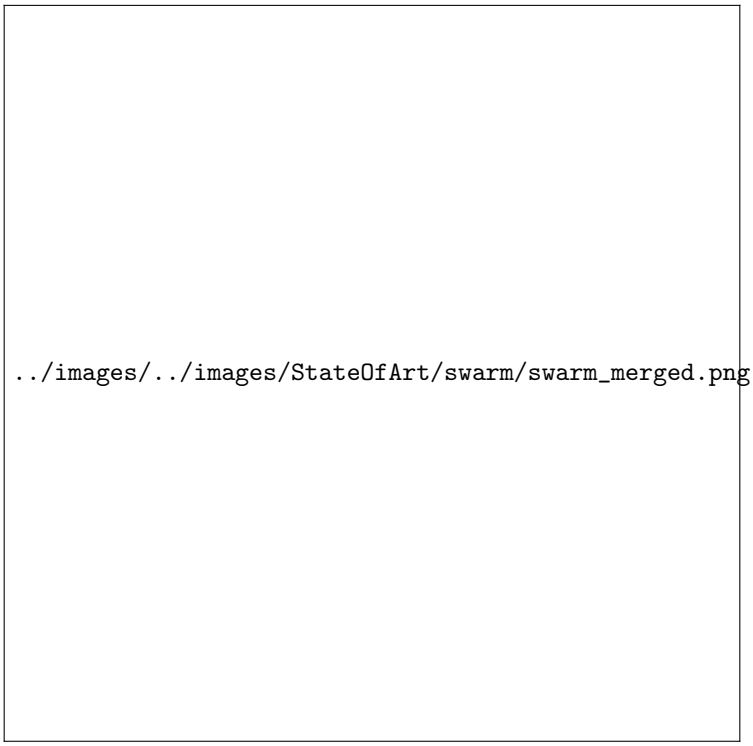


**Figure 2.1:** Foursquare Swarm show case

### 2.1.1 Dodgeball and Google Latitude

Dodgeball was a service that simply allowed users to share their current locations with friends through a map, and served as a precursor to Foursquare City Guide and Foursquare Swarm same functionality. Later was acquired by Google and rebranded as 'Google Latitude' [32] discontinued in 2013 and integrated into another discontinued social media by Google (Google+ [30]) and finally migrated into Google Maps 2.4 in 2018.

## 2.2 Snapchat

Snapchat [37] has been one of the most used social media platforms worldwide [36], mainly thanks to its instant messaging service and multimedia functionality, such as time-based content, which makes uploaded content visible only for 24 hours or visible only once to a user. This encourages users to check it daily in order not to miss anything and to keep up with relatives' lives. As a negative effect, these features have fueled some social impacts such as FoMO (Fear of Missing Out) [6, 12], leading users to become increasingly attached to the social platform and also creating self-comparison with content posted by other users. This problem is not strictly related to Snapchat, but is very common in every self-centric social media platform. Other concerns are raised by privacy and safety dued to the possibility to share current location into a geographic map [42], as done by Swarm and others (Section 2.1), Snapchat also enrich the map by showing geographic closer content to user, grouped into a unified cluster of information, as illustrated in Figure 2.2.

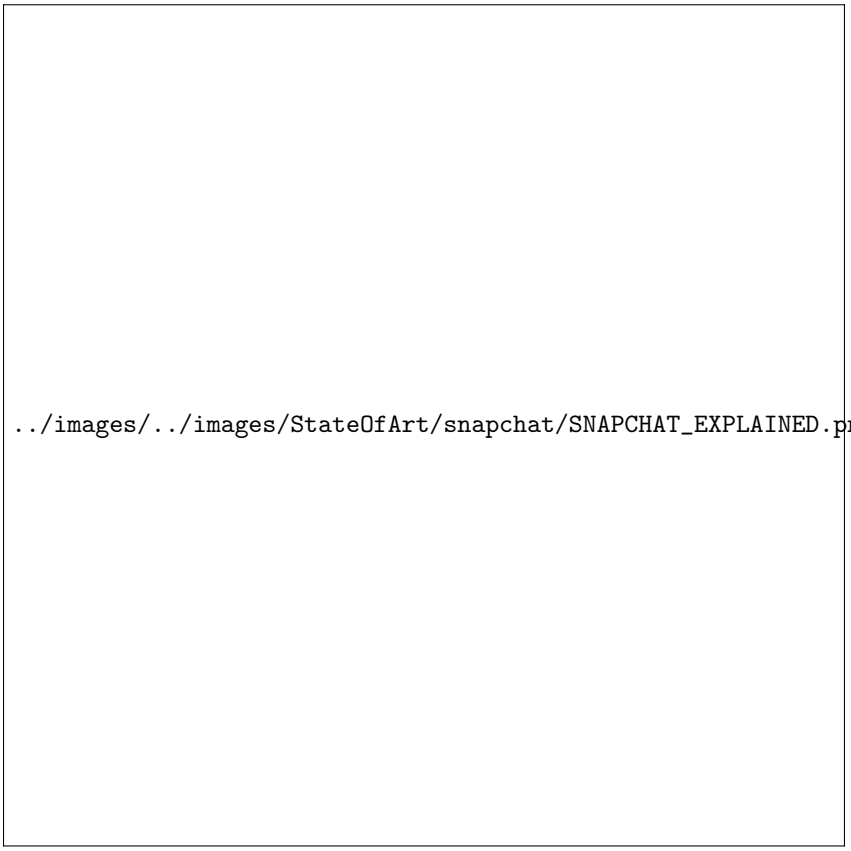


**Figure 2.2:** Snapchat functionalities

Moreover, as the Swarm app does 2.1, it allows users can log visited locations, including

public places such as bars, stores, and museums. This enables the association of user-generated content with specific points of interest (POI) and supports the aggregation of geographically and contextually related content to represent events, such as concerts or other real-word gatherings, as illustrated in Figure 2.3

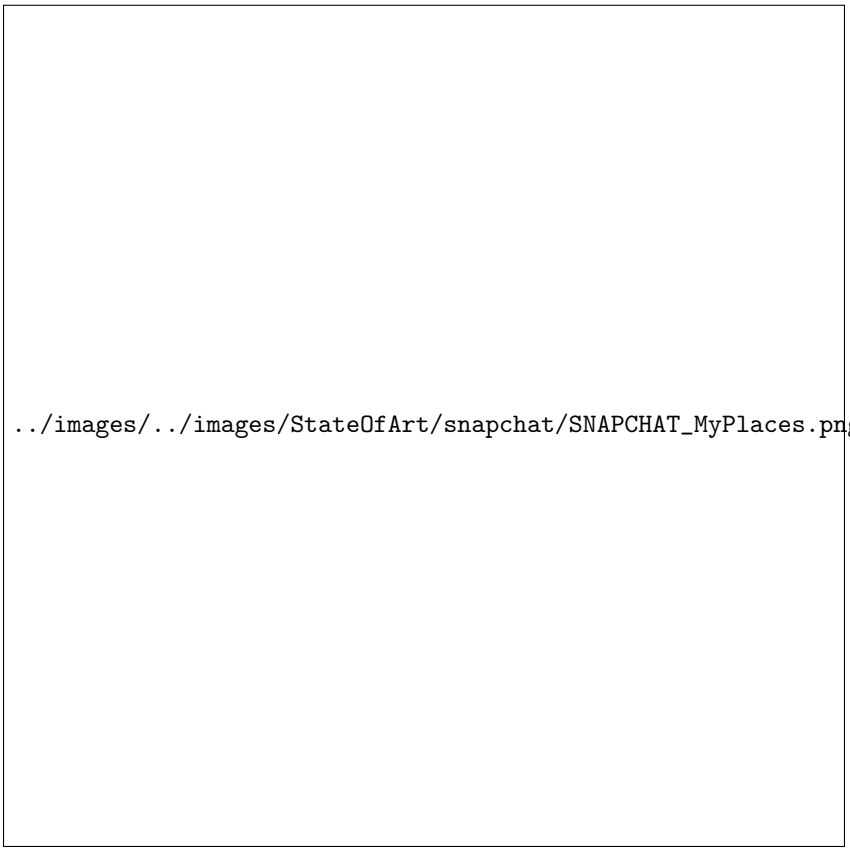


**Figure 2.3:** Snapchat my place functionality

Snapchat represents a notable example of the adoption of the "mobile-first" paradigm, expecially in social media applications. The platform leverages not only GPS module but also a variety of mobile device sensors, including the camera for augmented reality (AR) experiences and the microphone to records voice-based content. By integrating these hardware features into its core functionality, the app delivers a highly immersive and personalized user experience.

## 2.3 Instagram

Instagram, owned by Meta Platforms, is one of the biggest and most used social media of the latest decades. Through the introduction of numerous features and continuous innovations,

it has attracted more than two billion montly active users worldwide [48, 54].

The platform was initially released in October 2010 exclusively for iOS devices. In fact, at the time, the uploaded content was framed in a square (1:1) aspect ratio with a resolution of 640 pixels, reflecting the display characteristics of iPhones at the time. Later acquired by the formerly named Facebook Inc. (now Meta Platforms Inc.), the application was also deployed for Android systems in April 2012. Then, the original aspect-ratio limitations were removed, and several additional functionalities were introduced, including direct messaging and support for multiple images or videos within a single post.

A major development was the introduction of the "Stories" feature, which provide the same mechanism created by Snapchat 2.2, that allows users to publish content that remains visible for only 24 hours after publication. The addition of this ability contributed significantly to Instagram surpassing Snapchat in user engagement metrics by 2019 [57].

To enrich the media sharing, Instagram provides a variety of creative tools, including photographic filters, image-editing capabilities, and even a live-streaming functionality. The platform also popularized the use of hashtags as a content categorization mechanism, enabling users to discover media and connect with communities centered around shared interests and topics.

Instagram to encourage content discovery adopt a sophisticated recommendation systems that leverage contextual information, including geographic location derived from GPS or cellular network data. Such metadata contributes to the generation of personalized recommendations, allowing users to discover nearby profiles, locations, and content that align with their interests [35].

Despite its popularity, Instagram has been the subject of significant criticism. Concerns have been raised regarding user privacy, large-scale behavioral profiling, and the potential influence of recommendation algorithms on user behavior. Additional controversies involve the platform's impact on adolescent mental health, content moderation practices, and the dissemination of illegal or inappropriate content.

### 2.3.1 Instagram Map

Despite its numerous functionalities, Instagram introduced in 2022 a map-based content discovery feature[18]. By anchoring content to existing geographic entities such as cities or others POI, users can explore both nearby or remote content, as illustrated in Figure 2.4.

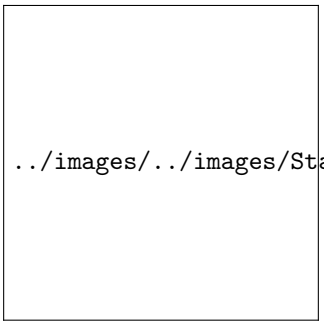


**Figure 2.4:** Instagram map discovery

In 2026, this functionality underwent a redesign: content is no longer presented as spatial clusters on the map (as implemented in Snapchat, Figure 2.3), instead is represented in a grid-based layout when a specific place is selected via the searchbar of the "Explore" section, as illustrated in Figure 2.5.

In addition, Instagram provides functionality for sharing real-time location with users connections (followers), similarly to other social media platforms discussed in Sections 2.2 and 2.1.


**Figure 2.5:** Instagram map content 2026

## 2.4 Google Maps

Google Maps[29] is one of the services provided by Google Inc. Initially released as a web-based platform and following the wide adoption of mobile devices, extended for both iOS and Android systems. The service provides satellite imagery, aerial photography, route planning and 360° interactive panoramic views, commonly known as "Street View"[31]. Google Maps consents users to explore their surroundings by displaying stores, points of interest, and attractions near to their current location or consnet to explorer it scrolling it. The application consents users to explore their surroundings by displaying stores, attractions or others Point of Interests near their current location, or by enabling them to explore other areas by scrolling the map. These points of interest can be organized into categories such as restaurants, coffee shops, and other predefined types as shown in Figure 2.6. For each place, users may submit reviews by commenting or uploading pictures and videos, thus to create a collaborative information ecosystem. Similarly to Foursquare Swarm 2.1, Google Maps also introduces gamification elements such as badges and user levels to incentivize contributions and improve the quality of place-based reviews.

Google Maps further provides APIs and libraries that allows the development of integrations to its services into thrid-party applications. In fact, the GeoMedia client app 5 through the React Native library adopted, renderized the Google Map as default map framework for Android devices. This service is also well integrated in its ecosystem and Android platforms as well, in fact, exists several standard features, for an instance 'recent places' for places visited by users (within their smartphone) or to have the routing to home or work, enabled and suggested when connected to the car or started in Android Auto [26].

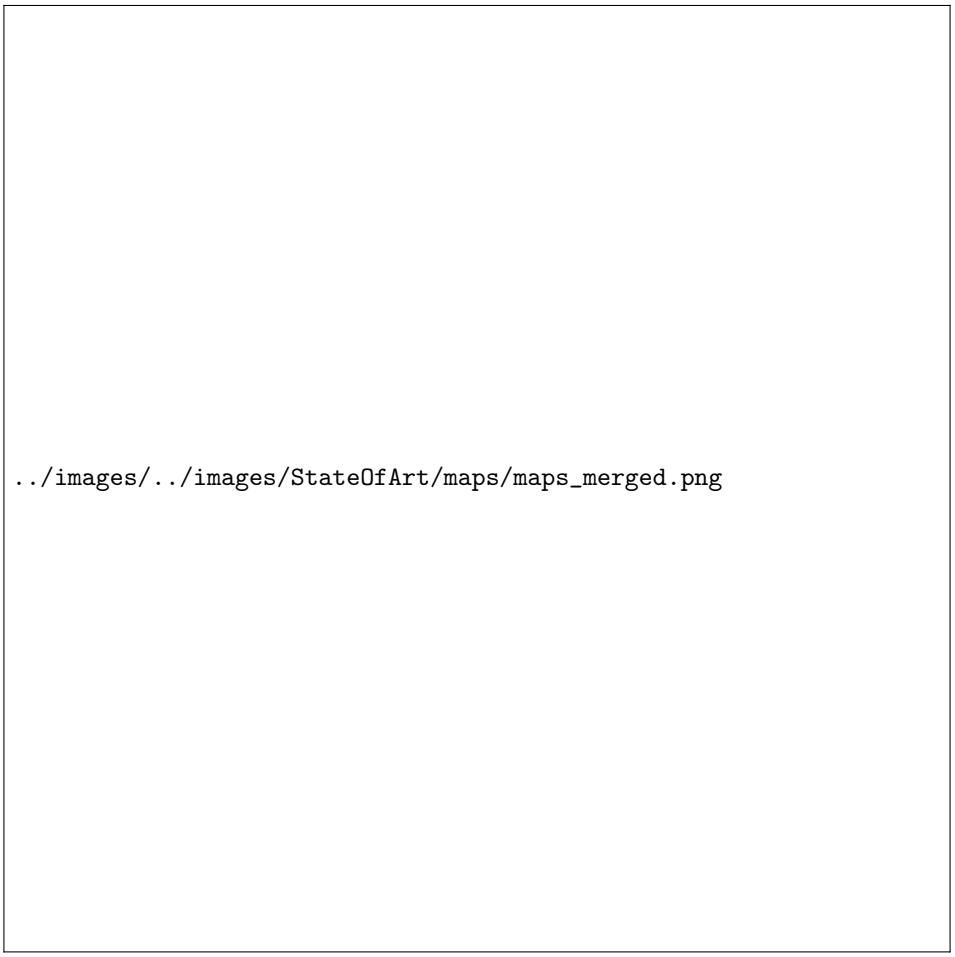


**Figure 2.6:** Google Maps show case

## 2.5 Others

There are several other social media platforms that integrate map features and geolocation capabilities. Although they are less popular than the platforms previously discussed, they still cover different scenarios and provide innovative functionalities. For example, Strava, originally developed as an application for tracking running and cycling activities, has evolved into a social networking, by allowing users to share their sports activities, connect with friends, and discover new training partners. While Strava uses geolocation primarily to support sports-related activities, other services adopt location-based interaction at the core of the user experience. More closely related to GeoMedia there are two social media platforms, Jagat [33] and Auglinn [24], both focusing on enabling users to share content and interact with others through a geographical map and current location.

### 2.5.1 Jagat

Jagat offers several functionalities and highly responsive performance, despite a difficult usability perspective for first-time users, which can appear crowded and confusing, as it displays too many buttons and pieces of information simultaneously, as can be seen in Figure 2.7.
Similar to Snapchat 2.3 or Swarm 2.1, Jagat provide a real-time map enriched with the current location of friends but with more metadata like users' movement speed, or time spent in a specific location or they device battery level. These features raise significant priacy and safety concerns due to the amount of personal information collected and shared.

As social media platform, it also enables users to upload photos and videos, not anchored to any specific location but whenever users want; functionality implemented also by GeoMedia.

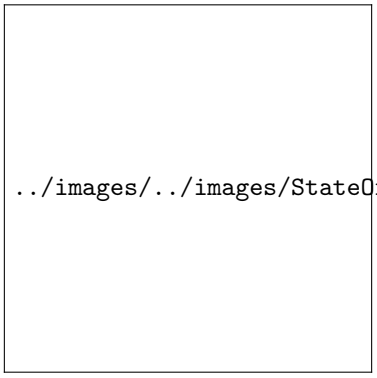


**Figure 2.7:** Jagatt map interface

### 2.5.2 Auglinn

Available for both Android and iOS platforms, and also for the web, Auglinn is the closest application to GeoMedia. It provides a platform that allows users to create messages enriched with multimedia content and share them with other users through a geographical map. Moreover, Auglinn also offers augmented reality (AR) functionalities to display location-based messages, which can be created either remotely or on demand through a mobile device. It also allows users to create different public, view-only, or private spaces, which act like containers for content (called "notes" or "boxes") and set an expiration time for them, as GeoMedia as well. Other users can respectivaly interact by commenting, saving, liking or sharing notes found.

An example of Auglinn usage is shown in Figure 2.8

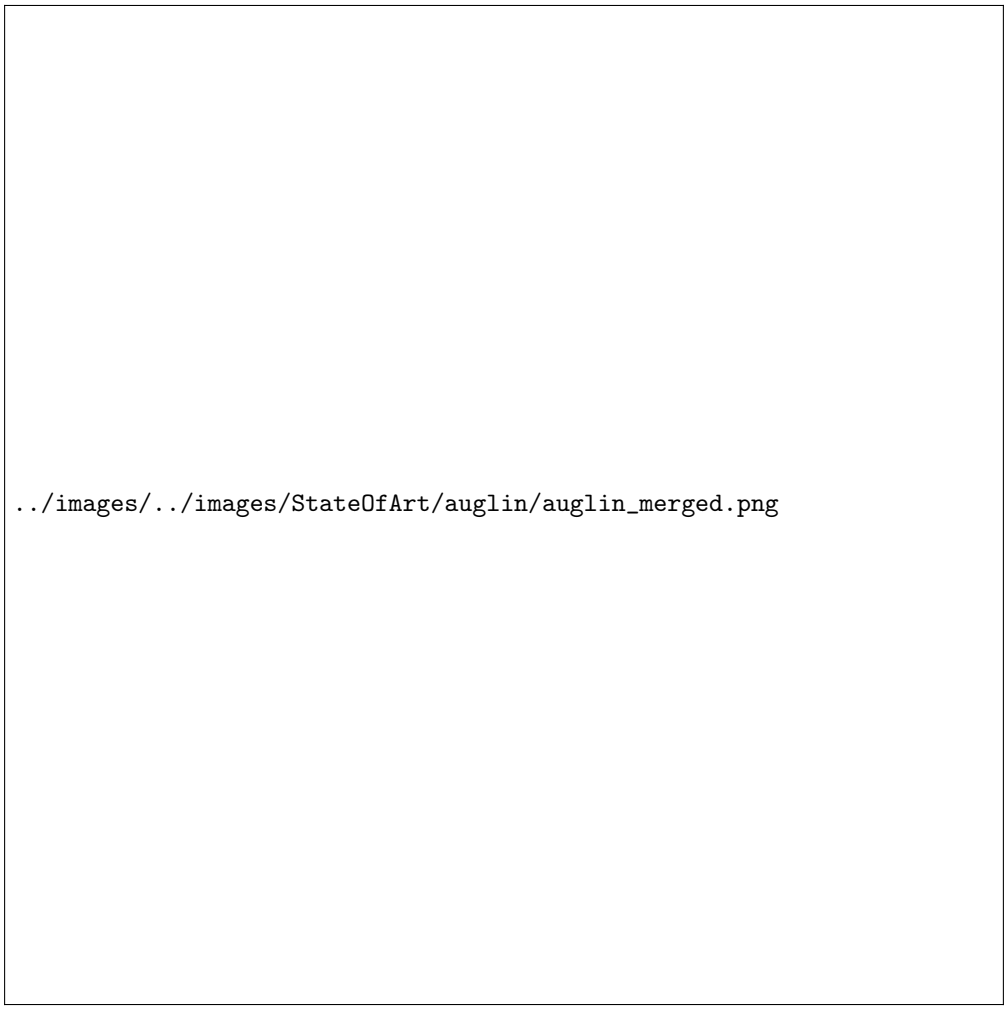


**Figure 2.8:** Auglinn show case

# Chapter 3

# GeoMedia

## 3.1 Concept

GeoMedia introduces a geographically oriented social platform in which users can upload and share content in the form of posts. Each post acts as a container for the information that users want to publish, enabling location-based interactions and content discovery.

The original concept focused exclusively on textual comments (called *GeoComments*), then extended to support any kind of multimedia content. Finally enriched with the concept of collections, which act as layers for group posts and provide more sophisticated and domain-specific use cases, allowing users to organize and restrict the scope of displayed content.

To realize the idea behind the application, an appropriate software architecture (later explained in Cap. 4) and different supporting technologies must be adopted. The client layer, which represent the primary point of interaction between users and the platform, is implemented as mobile application. This app allows user to discover the physical world while leveraging GPS capabilities of the device, thus to interact with the application layers and relative posts. Moreover, the mobile app needs to communicate with a backend, which works as coordinator for implementing the system‘s business logic and determining which content should be provided to the users’ requests (basing on their current position and other metadata).

## 3.2 Collections

Collections represent containers for posts that users can create and customize by assigning a title, an icon and a color, allowing them to be easily distinguished from other collections.

Users can also configure the visibility of each collection according to the following dimensions:

- **Users**: specifies which users are authorized to view the collection in the different sections of the application (e.g., the map view and the collections list).
- **Time**: restricts the visibility of the collection to a specific date range. Additionally, recurring visibility intervals can be configured on a monthly basis (repeating on the specified days of each month) or on a yearly basis (repeating during the specified month and days each year).

Additionally to defining who can view the collection, owners of a collection can specify which users are authorized to create post within it. This distinction between viewers and contributors provides finer-grained access control, enriching users experiences while enabling more effective moderation and content management.

For each collection, the owner can also enable or disable the *remote posting* feature. When enabled, users are allowed to manually select the geographical location associated with a new post, byu dragging a marked through the map. Instead, when disabled, posts can only be created at the user's current GPS location. This last case, makes the platform suitable for a wide variety of scenarios, such as restaurant reviews, traffic reports and real-time local information; avoiding the likelihood of unrelated or misleading content and strengthening the connection between the online content and the physical world.

Moreover, the creator of the collection can reorder the contained posts, enabling a sequential visibility order in which a post becomes visible only after the previous one has been viewed.

Figure 3.1 shows the explained functionalities with the respective order:

- The panoramic view of a single collection (top-left);
- The sequential ordering function (top-right);
- The application of date-time and recurrence limitations (bottom-left);
- The management of creators allowed within the collection (bottom-right).

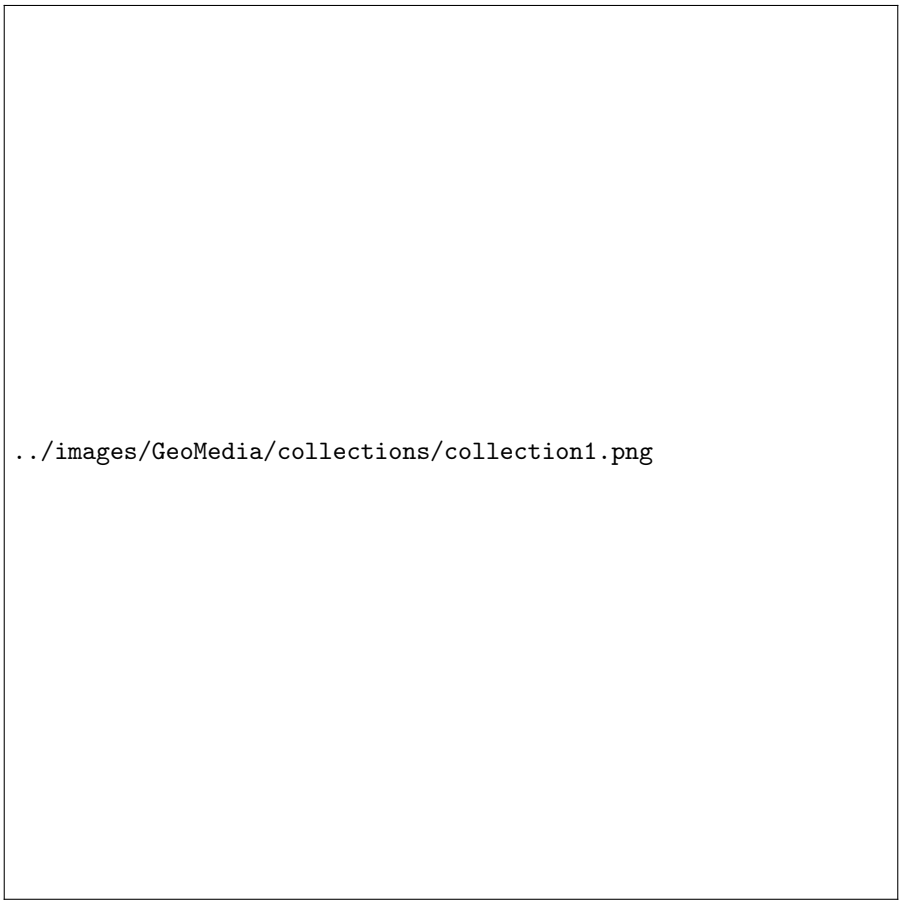


**Figure 3.1:** Collection panoramic and functionalities.

Finally, as shown in Figure 3.2 each collection can be associated with an unlimited number of hashtags, which serve as thematic labels for content classification. These hashtags facilitate users to discover topics and identify collections of specific interests, toward to create a personalized feed in the "For you" section of collections.

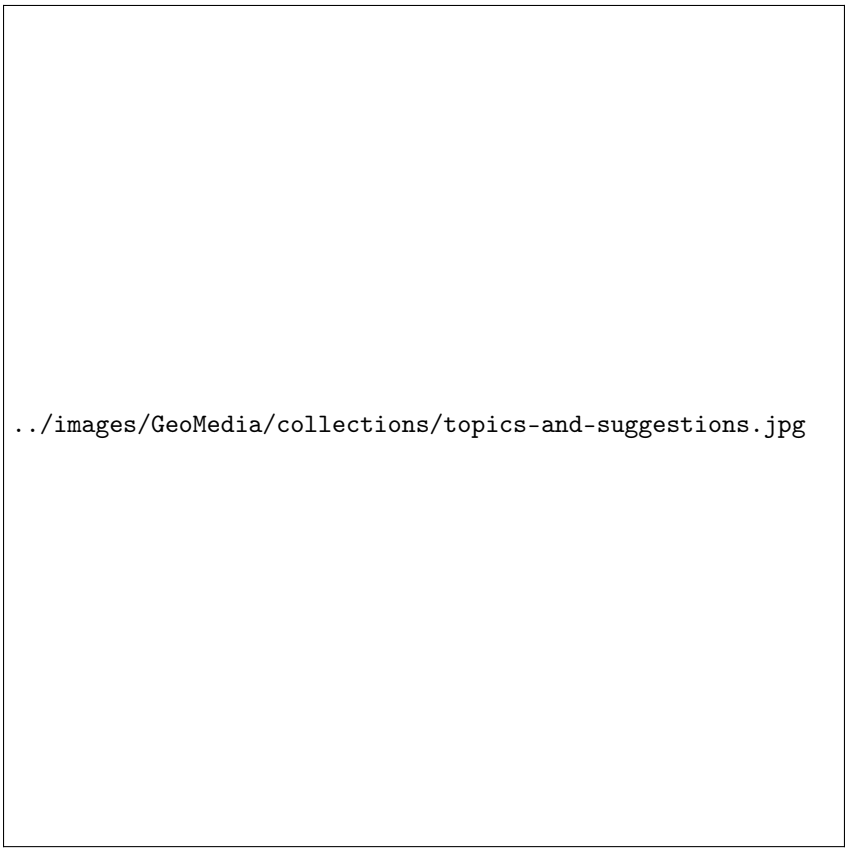


**Figure 3.2:** Collection topics and suggestions

## 3.3 Map

The **Map** section represents the core component of GeoMedia, providing an interactive geographical map enriched with location-based posts. Each post is represented by a marker positioned at the geographical coordinates where it was created or, when the associated collection allows *remote posting*, at the location selected by its creator.

To improve visual recognition, each marker adopts the color assigned to its collection, allowing users to associate the context to which the post belongs.
By selecting a marker, the complete content of the corresponding post is displayed and express appreciation by liking the post, or, whenever a post contains an attached media file, users can download it regardless of its file format. Furthermore, users, can view the creator's profile, checking its statistic displayed in graphs or browse other posts of the same creator which are accessible from current location (and others metadata). To preserve users' privacy, posts displayed in viewer mode are represented as lists and rather than on a map, thus to hide the positions of the creator and preserving the gamified experience of discovery

through the map. However, this feature provide a peek of some content of the user in way to provide users potential related posts.

As shown in Figure 3.3, the interface (described in Chapter **??**) has been designed to provide a simple and intuitive user experience. In particular, it highlights the most relevant collections available around the user's current location by prioritizing those containing the largest number of posts near the current location, this feature is called "local collections". Moreover, GeoMedia offer different modalities toward the discovery of collections, such as:

- **For you**: That provide a personalized feed for user, to discover collections based on their interests topic selected within the same page.
- **All**: Which provide all the collections in form as a list in random order.
- **Popular**: Represents a ranking of the most frequently used collections, computed based on the total number of posts associated with each collection.
- **Trending**: Represents a ranking of the most frequently used collections during the last four hours.

**Figure 3.3:** Map tab overview and collection discovery

GeoMedia also supports multiple styles among five different map themes, through the application settings, allowing the interface to better adapt to personal preferences and different usage scenarios. The current position of the user is continuously followed through the GPS capability thus to make appear and hide different posts as user moves through the physical world; then the map automatically shift its center to the current location (unless the user manually navigates the map).

### 3.3.1 Post creation

Through the map component, users can also access to the post creation section, where the following required information are presented:

- **Collection**: the collection to which the post will belong.
- **Title**: the title of the post.
- **Visibility Range (km)**: the geographical distance within the post is visible to other users (between *0.2 - 30.0 KM*).

- **Description (optional)**: a textual description with no practical limitation on its length.
- **Media (optional)**: allows users to attach one or more files from the mobile device, including images, videos, audio recordings or any other supported file type. Alternatively, the integrated camera can be used to capture a photograph and share it within the post.
- **Visibility Constraints**: optional restrictions that limit the accessibility of the post according to temporal conditions (a date-time interval with optional recurrence) or user-based permissions (specifying which users are allowed to access the post).

The geographical position of a post is automatically retrieved from the user's current GPS location. Each post is therefore characterized by its latitude, longitude, and altitude. Although altitude is not currently exploited by the platform, it is stored to support future extensions of the system.

### 3.3.2 Haversine formula

To determine which posts are visible to a user, GeoMedia evaluates the geographical distance between the user's current position and the location associated with each stored post. Only posts whose visibility radius includes the user's current position are returned by the server and displayed on the mobile devices.

The server computes these distances using an implementation of the Haversine formula[13]. Specifically, the implementation, reported in Listing 3.1, calculates the great-circle distance between two points on the Earth's surface from their latitude and longitude coordinates. The resulting distance is then compared with the visibility radius associated with each post to determine whether the post is within the user's area of visibility.

The Haversine formula models the Earth as a sphere and provides an efficient approximation of the distance between two geographic coordinates. Although ellipsoidal models such as Vincenty's formulae[14] provide higher accuracy, overall the precision offered by the Haversine formula is sufficient for this application, where the visibility of a post depends on whether a user is located within a predefined proximity range. Furthermore, its low computational complexity makes it particularly suitable for backend applications requiring frequent geographical distance calculations.

For short-range distance calculations, the error introduced by the spherical approximation is negligible compared with the uncertainty of standard GPS positioning. At distances of

approximately 30 km, the deviation from an ellipsoidal model is generally within a few meters, while for distances below a few kilometers it is typically reduced to the meter or sub-meter range.

```
# Haversine formula implementation
# Compute the geodesic distance between two geographic coordinates
# using Vincenty's inverse formula.

# Args:
#     areaKM (float): The range within the post would be visible
#     post_lat (float): Latitude of the post.
#     post_lon (float): Longitude of the post.
#     curr_lat (float): Current latitude coordinate of the user.
#     curr_lon (float): Current longitude coordinate of the user.

# Returns:
#     bool: True if the user position is included within the area of visibility of
    the post.

def check_post_area(areaKM, post_lat, post_lon, curr_lat, curr_lon):
    from math import cos, asin, sqrt, pi

    post_lat = float(post_lat)
    post_lon = float(post_lon)
    curr_lat = float(curr_lat)
    curr_lon = float(curr_lon)



    dLat = (post_lat - curr_lat) * pi / 180
    dLon = (post_lon - curr_lon) * pi / 180

    a = (0.5
        - cos(dLat) / 2
        + cos(curr_lat * pi / 180)
        * cos(post_lat * pi / 180)
        * (1 - cos(dLon)) / 2
    )
```

```
33
34      d = round(6371000 * 2 * asin(sqrt(a)))
35
36      return d <= areaKM * 1000
```

**Listing 3.1:** Python implementation of the Haversine-based visibility check.

## 3.4 Exclusivity

The exclusivity feature assume an important role in GeoMedia, as it allows the application to be adapted to different use cases and real-world scenarios by providing fine-grained control over content visibility.

As previously described, collections act as containers for posts. Consequently, posts inherit the visibility constraints defined at the collection level and may additionally introduce narrower restrictions. In other words, if a user cannot access a collection, they cannot access any of the posts contained within it. However, access to a collection does not automatically imply access to all its posts, as individual posts may apply additional limitations as:

- **Geographical range**: each post defines a visibility radius, which determines the maximum distance from its location at which it can be accessed. The supported range goes from 0.2 km up to 30 km.
- **Collection-based limitations**: posts inherit the restrictions defined by their parent collection, including:
  - **User limitation**: defines which users are allowed to access the collection.
  - **Time limitation**: defines the period during which the collection is visible, consequently the possibility to access to the post.
- **Post-specific time limitation**: applies temporal restrictions directly to a single post, narrowing the visibility inherited from the collection.
- **Post-specific user limitation**: restricts access to a single post for a specific group of users, narrowing the permissions inherited from the collection.
- **Sequentiality limitation**: collections that enable sequential access expose only the next available post (or the first post when starting the sequence) according to the

order defined by the creator. This mechanism allows creators to guide users through a predefined path or experience.

### 3.4.1 Use cases

By combining geographical positioning, collections, access restrictions and sequentiality, GeoMedia supports different scenarios and applications. Some examples are described below:

- **Local tours**: Through collections, which act as containers for multiple related posts, users can select specific layers created by a guide or organization and access only the content belonging to that experience. Furthermore, the sequentiality feature can enrich guided tours by introducing a predefined order, allowing the next attraction or point of interest to become available only after the previous one has been visited.
  Posts can be created remotely by different authors (such as professional guides) and promoted across related collections through the use of hashtags. As a further development, this functionality enable a monetization mechanisms for the platform, allowing creators to provide exclusive paid experiences directly through the platform.

- **Recurrent posts**: GeoMedia allows users to define post visibility not only through geographical constraints but also through temporal conditions. A post can be associated with a specific date and time interval and can optionally be configured with a recurrence pattern, such as monthly or yearly availability. This enables scenarios where the same content becomes available periodically without requiring manual recreation, relevant for scenario such as anniversaries, birthdays or recurrent celebrations.

- **Sequential posts**: Collections with sequentiality enabled allow creators to define a specific order in which posts should be discovered. Only the first available post is initially displayed, while subsequent posts become accessible only after seen the previous one and satisfying visibility conditions defined by their geographical position, time constraints, and user permissions.
  This mechanism introduces gamification possibilities within the social platform, enabling experiences such as digital scavenger hunts, interactive tours, treasure hunts or location-based challenges, where users progressively unlock new content by exploring the real world.

- **Augmented Reality**: GeoMedia does not currently implement Augmented Reality (AR) functionality; however, it includes an in-app camera component that can be lever-

aged for future AR-based enhancements. This feature could enable the integration of digital information above the real-word environment, visualizing for example historical information or media content. With this development, the mobile application would improve the user experience by providing a more immersive and interactive way to explore and interact with geographically referenced content.

An interesting scenario can be adopted by the City of Padua, thus to illustrate through the app one of its well-known sayings: "città dei tre senza" (literally, "the city of the three withouth"). This expression refers to three notable things that Padua has-but or hasn't in a metaphorical way:

- **The lawn without grass**: referring to *Prato della Valle*, which despite its name ("meadow of the valley"), feature ornamental grass and is not a lawn in the traditional sense;
- **The saint without a name**: referring to *Sant'Antonio da Padova* who was actually born in Portugal, and only spent the last years of his life in Padua.
- **The café withouth doors**: referring to one of the most historic Italian café, *Caffé Pedrocchi*, which during wartime remained with doors opened all day and night to provide shelter.

In this case, digital markers at these location as shown in Figure 3.4, enable the storytelling of each unique characteristic, by uploading historical photograph or facts.

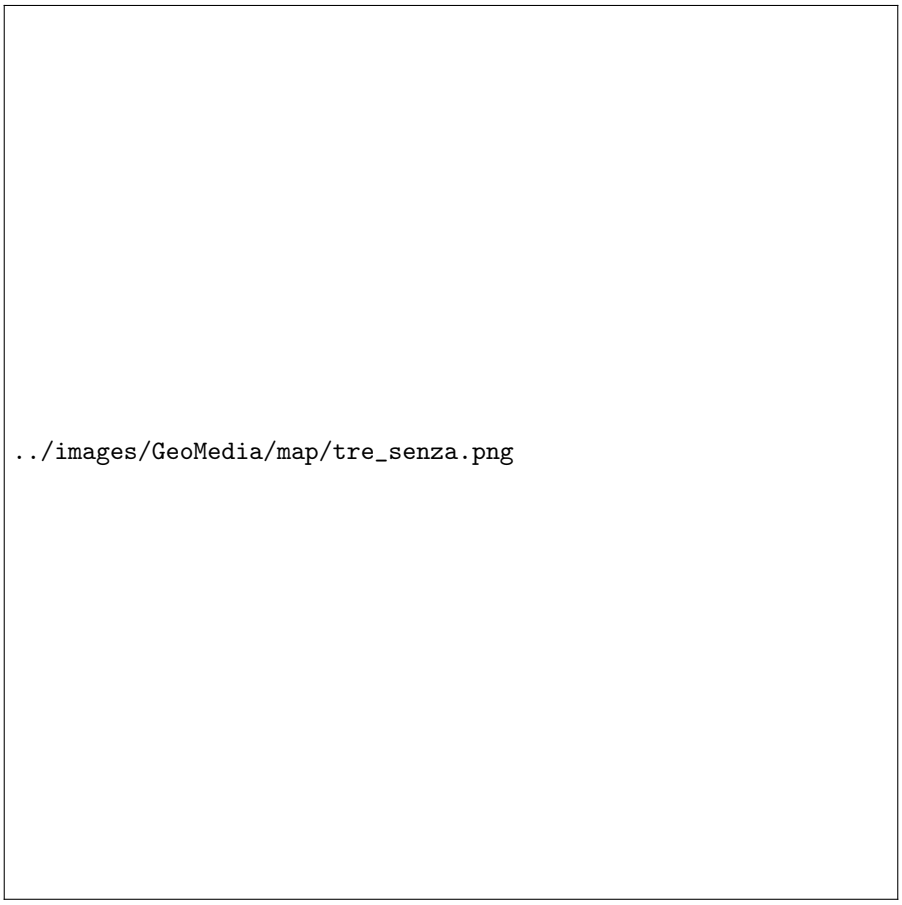


**Figure 3.4:** City of Padua use case for its saying of "Tre senza".

## 3.5 User

Even if not profile-centric, GeoMedia, allows users the ability to personalize their profiles by specifying basic personal information, such as their first and last name but also to upload a profile picture. These features allow users to establish a recognizable identity within the platform and enhance the overall user experience.

In addition, the GeoMedia mobile application provides users with infographic-based statistics that summarize their activity on the platform, such as the number of published posts distributed across months or which in which collections they are active as pie-chart. As shown in Figure 3.5, these visual summaries offer users an immediate overview of their contributions and engagement within the application.

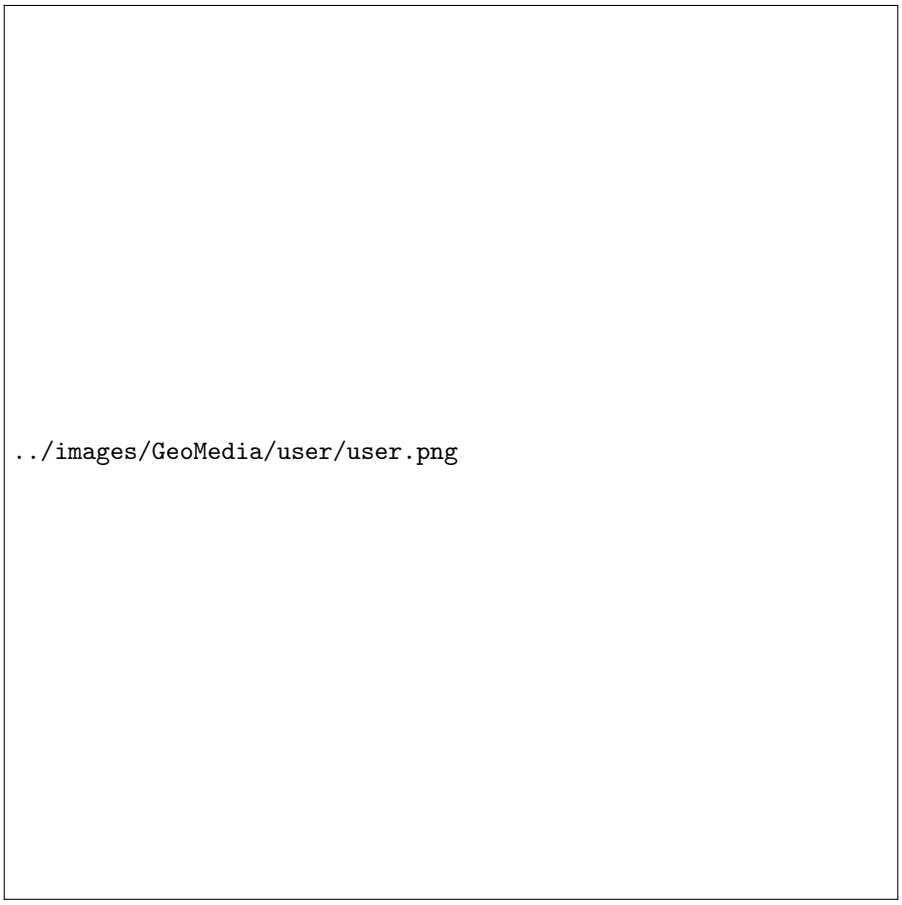


**Figure 3.5:** User profile editor and viewer mode.

## 3.6 Functional comparison

While GeoMedia incorporates functionalities commonly found in existing social media platforms presented in Chapter 2, it extends their capabilities through several distinctive features. Table 3.1 provides a comparative overview of the supported functionalities across the analyzed platforms.

| *Feature* | GeoMedia | Jagat | Aulinn | Foursquar Swarm | Snapchat | Instagram | Google Maps |
|---|---|---|---|---|---|---|---|
| Remote location posting | ✓ | ✓ | ✓ | ✓ | – | ✓ | ✓ |
| Location-based restrictions | ✓ | – | – | – | – | – | – |
| Time restrictions | ✓ | – | – | – | ✓ | ✓ | – |
| Sequential posts | ✓ | – | – | – | – | – | – |
| Multi-author collections | Selective contributors | – | – | – | – | Everyone | Everyone |
| User-based restrictions | Selective viewers | – | – | – | Selective viewers | Selective viewers | – |
| Topic-based content | ✓ | – | – | ✓ | – | – | ✓ |
| User statistics | ✓ | – | – | ✓ | – | – | ✓ |

**Table 3.1:** Functional comparison of GeoMedia and related social media platforms.

# Chapter 4

# Architecture and Software design

The software architecture to concretize the concept behind GeoMedia, is represented by a three tier solution, which divide the whole platform into different actors according they operability and functions.

In specific, a three tier model is composed by:

Generally, a three-tier architecture consists of the following layers:

- **Client layer (Presentation layer):** which provides the user interface and manages user interactions. It is responsible for presenting data to the user and forwarding user requests to the application layer while remaining independent of the underlying business logic and data storage. *In GeoMedia, this is represented by the mobile app*

- **Application layer (Business Logic layer):** that implements the core functionalities of the platform and applies the business logic to process user requests. This layer acts as an intermediary between the presentation and data layers, encapsulating the application's logic and coordinating the exchange of data to ensure consistency and proper execution of system operations. *In GeoMedia, this layer is represented by the server-side component responsible for processing client requests, executing the application logic and coordinating communication between the client and the data layer through a RESTful interface.*

- **Data layer (Persistence layer):** that is responsible for managing data storage and performing data operations such as the creation, retrieval, update and deletion (CRUD) of persistent data. It provides an interface to databases or other storage systems while abstracting and encapsulating the underlying data access details from

the upper layers. *In GeoMedia, this layer is represented by the Database Management System (DBMS) that provides a full solution for data handling.*

This solution provide a classic and simple architecture, by exploiting technology functionalities such Network communication with TCP/IP stack, these actors are able to establish a communication among each other and then to exchange information and activate business process.

The UML schema in the Figure 4.1 can represent graphically the architecture established.

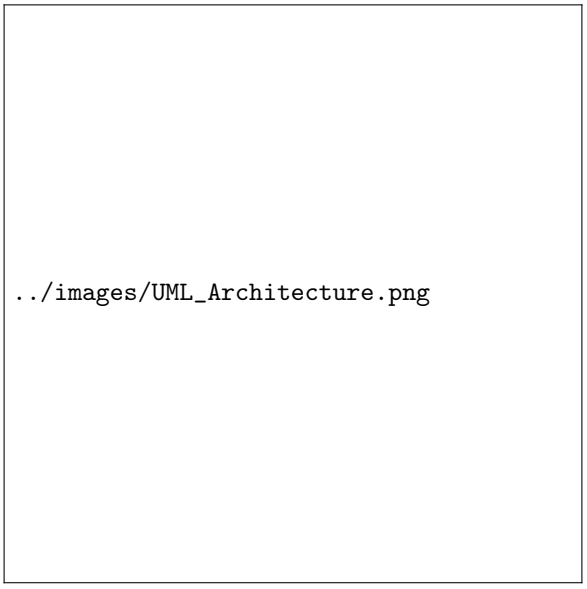


**Figure 4.1:** Three-tier Software Architecture UML.

Regarding GeoMedia, the communication is implemented through the HTTP protocol, which is one of the most popular and widely used protocols in the last decades. HTTP is a request-response protocol based on the client-server model that create a virtual link between the application layer (as server) that returns a response to the client (presentation layer) with the data requested. The data, referred to as packets, can be manipulated through intermediary network nodes (proxy servers, NAT, web caches, etc.) while respecting the TCP/IP structure. In order to ensure the confidentiality and security of the data, HTTP allows the adoption of SSL certificates (HTTPS), making the communication encrypted with the public key of the server (and decipherable only with the secret key available only to the application layer).

Instead, to establish the connection between the others layer (application and data) the protocol depends on the data layer, in the case of GeoMedia the DBMS is represented by Microsoft SQL Server (MSSQL) that operate through the TCP/IP stack with TDS (Tabular

Data Stream) protocol.

## 4.1 Microservices comparison

Another valid alternative to the architecture design adopted is the microservices structure, that see the system decomposed into a set of small and independent services, each responsible for a specific business capability.

Each microservice encapsulates its own business logic and expose its functionalities through interfaces, usually providing RESTful APIs. In fact, this design concerns mainly the backend layers such as application and data more than the client which still represent the single point of interaction with the users.

This design compared to the a monolithic solution provides high degree of modularity and flexibility and others several key advantages, such as:

- **Scalability:** individual services can be scaled independently according to their workload, reducing infrastructure costs and improving resource utilization.
- **Maintainability:** the decomposition into smaller services simplifies both code maintenance than testing and debugging, as each service has a limited and well-defined responsibility.
- **Technology independence:** each service can be implemented using the programming language, framework or database that best fits its operability, while respecting the compatibility through communication interfaces
- **Fault isolation:** failures occurring in one service are less likely to compromise the entire application, increasing the reliability of the system.
- **Continuous deployment:** updates can be released independently for each service, enabling faster development cycles and reducing deployment risks.

Despite these advantages, the adoption of a microservices architecture represent an additional complexity which can be avoided for simple projects or applications with low usage. In fact, the challenge related to network communication, together with issues such as data confidentiality and authentication, distributed data transactions or service orchestration tend to outweigh the architectural benefits.

The three-tier architecture designed for GeoMedia represents a monolithic solution that is sufficient for the current size and usage of the application, successfully supporting

its functional requirements. Nevertheless, the clear separation between the presentation, application and data layers ensures that the software remains extensible and can progressively evolve towards a microservices architecture without requiring a complete redesign.

On the other hand, the adoption of a microservices architecture could be an interesting and justified scenario to support local and distributed databases across different countries. Since GeoMedia focuses on local data, synchronization would only be required for user accounts and collections, while posts could be stored on country-specific servers according to their geographical coordinates metadata.

A concept of the microservice adoption for GeoMedia is shown in the UML in Figure 4.2


**Figure 4.2:** Microservices Software Architecture UML.

# Chapter 5

# Mobile app

In order to provide its core functionalities, GeoMedia relies on mobile devices, such as smartphones and tablets, where GPS capabilities and internet connectivity can be leveraged to enable the discovery of the surrounding environment. Through the mobile application, users can visualize nearby posts based on their location and contribute new content by capturing images using the integrated camera. Therefore, the mobile application represents the most important component within the entire framework. Furthermore, its development introduces additional design challenges related to User Experience (UX), usability, performance limitations of mobile devices and the protection of user privacy.

## 5.1 React Native framework

React Native[51], commonly abbreviated as RN, is an open-source software development framework created by Meta Platforms (formerly Facebook Inc.) for building cross-platform mobile and lately even desktop applications. Unlike traditional development approaches, which require separate codebases for different operating systems, React Native enables developers to maintain a single codebase that is transformed and adapted during the deployment phase toward the destiantion platform. This approach allows the reuse of all or at least the most of the application logic across different platforms, such as Android and iOS. Consequently, it reduces development and maintenance efforts while preserving the performance and user experience expected from native applications.

Under the hook, React Native relies on a component-based architecture, where user interfaces are defined through reusable React components. React itself is a framework originally designed for web application development, introducing an efficient rendering

mechanism that updates only the components affected by changes in application state, avoiding unnecessary re-rendering operations. React Native extends this concept by enabling communication between Javascript/TypeScript based components and native platform APIs, allowing applications to access specific peripherals like GPS, cameras, haptic feedback actuators or several other embedded sensors and functionalities. More specifically, React Native consents to write application logic using web-oriented programming languages, such as JavaScript or TypeScript, while adopting a styling approach similar to CSS for interface design. UI elements are represented through reusable components, which can be considered analogous to HTML elements in traditional web development. Finally, during the deployment process, these components are translated into their corresponding native platform components, this enable the app to achieve a native-like user experience while maintaining a shared development environment.

For GeoMedia, React Native represents a suitable technological option, allowing the platform to reach both Android than iOS markets. Overall, React Native is well supported by various components and libaries that can fully satisfy the requirements of the service.

### 5.1.1 Ionic Framework comparison

A first version of GeoMedia was been developed with IonicFramework[45], which is a valid framework for web development with the capability to encapsulating the web application into a native one that will render and interpretates the content as browsers do.

Moreover, as advantages, Ionic allows developer to develop with different web framework such as: ReactJS, AngularJS, VueJS and integrate the Ionic components.

A first version of GeoMedia was developed using Ionic Framework[45], which is a valid framework for web development with the capability of encapsulating web applications into mobile applications. This approach could seem to be similar to ReactNative but it's core is totally different, since IonicFramework rerender and interpretate the components on running time, as web browsers process web pages, while ReactNative compile and traduce into native components providing higher performance and more optimal resource utilization.

Moreover, one of the main advantages of Ionic is the possibility for developers to work with different web frameworks such as ReactJS, AngularJS or VueJS, while integrating Ionic's components.

A visual comparison between the Ionic previous version of GeoMedia and the React Native final version is shown in Figure 5.1.


**Figure 5.1:** GeoMedia UI comparison between the Ionic Framework version (above) and the React Native version (below).

### 5.1.2 Libraries

One of the main strengths of React Native is its extensive ecosystem of free and open-source libraries and components. These resources are widely documented and maintained by the community, with the official React Native Directory [50] serving as the primary reference for discovering and evaluating compatible packages.

Overral GeoMedia for the Android version developed use several dependencies packages such as:

- **@gorhom/bottom-sheet**: provides customizable bottom sheet components that can be expanded or collapsed to display additional content without leaving the current screen, used mainly in the map component to render collections.
- **@react-native-community/geolocation**: provides access to the device's geolocation services, enabling the application to retrieve the user's current position.
- **@react-native-community/slider**: offers a slider component that allows users to select value within a specified, applied for chose the distance range in the post visibility.

- **@shopify/flash-list**: offers a high-performance list component optimized for rendering large collections of data with improved memory usage and scrolling performance.
- **expo**: provides the development framework and runtime environment for building, testing and deploying React Native applications, together with a wide range of native APIs.
- **expo-router**: implements a file-based routing system that simplifies navigation between the different screens of the application.
- **expo-secure-store**: securely stores sensitive information, such as user credentials, by leveraging the secure storage facilities provided by the underlying operating system.
- **react-native-gesture-handler**: enables advanced gesture recognition, including swipes, taps and drag interactions, providing a more responsive and native user experience.
- **react-native-gifted-charts**: provides customizable chart components for visualizing statistical information about the usage on the user profile.
- **react-native-maps**: integrates interactive maps into the application, allowing the visualization of geographic information and user locations.
- **react-native-reanimated-carousel**: implements an animated carousel component for displaying file attached into a post through horizontal scrolling.
- **react-native-share**: through the native sharing facilities of the operating system, allows to share or store the media attached in a post such as images or documents-
- **react-native-sortables**: provides sortable list components that allow users to reorder items through drag-and-drop interactions, used to arrange the post whenever th esequentals features is active in a collection.
- **react-native-vision-camera**: provides high-performance access to the device's camera, enabling image capture thus to attach piuctures in the post withouth levaing GeoMedia.

A particular emphasis should be placed on libraries that provide access to device APIs and hardware features, such as sensors and other physical components.

### Map & GPS

The map is the core component of GeoMedia and is implemented using the 'react-native-maps' library, which during the compilation process is translated into the corresponding native implementation for each platform, namely Google Maps on Android and Apple Maps on iOS.

On the Android platform, the Google Maps component provides a wide range of features. These include graphical customization of the map through styling properties, as well as the display of commercial, historical or other points of interest, such as coffee shops, restaurants or retail stores. In GeoMedia, these elements remain visible and serve as useful landmarks, helping users orient themselves within the displayed environment and among the posts published through the app itself. In fact, the map supports the rendering of additional geographical elements with specific coordinates, known as markers. Each marker acts as a visual placeholder that identifies the physical location associated with a post.

In addition, the library exposes several callback functions and hooks that allow developers to implement custom logic. One such feature is continuous background user location tracking, which GeoMedia leverages to dynamically load location-aware posts as the user moves through the physical environment. This approach ensures that the displayed content remains relevant to the user's current location. To optimize network usage and avoid unnecessary server requests caused by minor location variations, GeoMedia applies a 50-meter movement threshold, triggering a new request and post update only when the user's displacement exceeds this distance.

In order to access Google Maps services on Android, a Google Maps API key is required, which serves as an authentication token that links the application to a Google Cloud Project (a Google suite that offers access to other products such as shared services, computing resources, and others), enabling access to the Google Maps SDK and its associated services. The API key follows the pricing policy defined by Google. Specifically, it applies a 'pay-as-you-go' pricing model[20], in which the first 10,000 map loads (renderings) per month are free, while each additional 1,000 map loads costs 7 $.

An implementation of map renderig is shown in Listing 5.1

```
<MapView
        ref={mapRef}
        style={styles.map}
        showsUserLocation={true}
        toolbarEnabled={true}
```

```
        followsUserLocation={true}
        initialRegion={{
            latitude: UserPosition.latitude,
            longitude: UserPosition.longitude,
            latitudeDelta: 0.1,
            longitudeDelta: 0.1,
        }}
        customMapStyle={
            mapPreferenceStyle == "system" ? //here thus to render automatically when toggled by user via status-bar
                colorScheme == "dark" ? MapDarkMode : MapLightMode
                :
                mapPreferenceStyle
        }
        onUserLocationChange={(event) => {
            const { latitude, longitude } = event.nativeEvent.coordinate;

            if (positionChanged(latitude, longitude)) {
                setUserPosition({ latitude: latitude, longitude: longitude });
                get_posts_map({ latitude: latitude, longitude: longitude }) //retrieve new posts
            }
        }}
        zoomEnabled={true}
        camera={{
            center: {
                latitude: UserPosition.latitude,
                longitude: UserPosition.longitude,
            },
            pitch: 30, // like the angle
            heading: 0, // direction the camera faces (0 = north)
            zoom: 13.7, // optional, overrides altitude
        }}
        pitchEnabled={true}
        showsCompass={false}
        showsBuildings={true}
        showsMyLocationButton={false} //make it custom thus to give space to
```

```
    collections
41  >
42      {//rendering the markers
43          postMarkers?.map((p, index) => (
44              <Marker
45                  key={p?.ID}
46                  coordinate={{
47                      latitude: p?.LATITUDE,
48                      longitude: p?.LONGITUDE,
49                  }}
50                  pinColor={p?.COLOR}
51                  title={p?.TITLE}
52                  onPress={() => { ///redirect to post viewer
53                      router.push({
54                          pathname: 'viewer/PostViewer',
55                          params: {
56                              postid: p?.ID,
57                          }
58                      });
59                  }}
60              />
61          ))
62      }
63  </MapView>
```

**Listing 5.1:** Implementation of MapView component for rendering posts.

Along with the map rendering, the integration of the GPS module provides the current coordinates of the user:

- **Longitude**: the angular distance east or west of the Prime Meridian, used to identify the horizontal position on the Earth's surface.
- **Latitude**: the angular distance north or south of the Equator, used to identify the vertical position on the Earth's surface.
- **Altitude**: the vertical distance of the user above mean sea level, estimated by the GPS sensor.

Even though altitude is not currently used by GeoMedia, it is stored for future developments.

This set of information is repeatedly used throughout GeoMedia, both for displaying nearby posts and during the post creation process, where the user's current position is automatically assigned as the post location. Specifically, when the current position is used, the altitude is also stored. Conversely, when the location is selected remotely, the altitude is not available. This is because the altitude can be estimated by the GPS sensor, whereas the altitude of a manually selected location cannot be reliably determined, as different points may correspond to buildings with multiple floors, skyscrapers, or mountainous areas.
In order to access device resources and user data, Android (as well as the iOS platform) adopts a permission-based security model. This requires users to explicitly grant permissions before applications can access protected system features, hardware components, and sensitive information.

An implementation of position retrieving is shown in Listing 5.3

```
1
2  async function getLocation() {
3         const hasPermission = await requestLocationPermission();
4
5         if (!hasPermission) {
6             Alert.alert(langselected?.permission.location.denied);
7             return;
8         }
9         if (Platform.OS === 'ios') {
10            Geolocation.requestAuthorization();
11        }
12
13        Geolocation.getCurrentPosition(
14            position => {
15                const { latitude, longitude } = position.coords;
16                if (positionChanged(latitude, longitude, UserPosition)) {
17                    setUserPosition(prev => ({ ...prev, latitude: latitude,
    longitude: longitude }));
18                    get_posts_map({ latitude: latitude, longitude: longitude })
19                }
20
21                // set map with center on user location
22                mapRef.current?.animateToRegion({
23                    latitude,
```

```
                    longitude,
                    latitudeDelta: 0.03, //zoom
                    longitudeDelta: 0.03, //zoom
                }, 1000);
                return { latitude: latitude, longitude: longitude }
            },

            ...

}
```

**Listing 5.2:** Location retriving snippet.

### File sharing & camera access

Another external implementation involving device sensors is camera access. This functionality, which is always granted through the permission model, allows GeoMedia to access the smartphone camera capabilities in order to capture images and upload them during the post creation process.

To accomplish this objective, GeoMedia integrates the third-party library 'react-native-vision-camera'. This library provides a set of hook functions that encapsulate the underlying camera access logic and simplify the management and storage of captured images, as shown in Listing **??**.

```

export default function OpenCamera(
    { onClose, storePhoto }: { onClose: () => void; storePhoto: (b64: string) =>
    void }
) {
    ...
    return (
        <View style={StyleSheet.absoluteFill}>

            <Camera
                ref={camera}
                style={StyleSheet.absoluteFill}
                device={device}
                isActive={true}
```

```
                photo={true}           // enable photo capture
                video={false}          // enable if you want video later
                enableZoomGesture={true}
                torch={flash === 'on' ? 'on' : 'off'} // continuous torch (flash preview)
            />

            {/* FLASH/REVERSE BUTTONS */}
            ...

            {/* CAPTURE BUTTON */}
            <View style={styles.controlsBottom}>
                <TouchableOpacity style={styles.captureButton} onPress={async () => {
                    let b64 = await takePhoto();
                    storePhoto(b64)
                }}>
                    <View style={styles.captureInner} />
                </TouchableOpacity>
            </View>
        </View >
    );
}
```

**Listing 5.3:** Location retriving snippet.

Alternatively, users can upload different types of files without restrictions on their respective extensions. GeoMedia supports various file formats, including audio files and PDF documents. To enable this functionality, the application allows users to select files directly from the device file system through libraries that interact with the operating system using well-defined system APIs. For this functionality, no specific visual components are required; instead, the file selection mechanism is triggered programmatically through a method call associated with an arbitrary button, which invokes the native file selection interface provided by the platform, as shown in Listing 5.4.

```

    async function file_pick() {
        try {
```

```
            const result = await DocumentPicker.getDocumentAsync({
                type: "*/*",
                multiple: true,
                copyToCacheDirectory: true,
            });

            if (result.canceled) return;

            let files = []
            for (const asset of result.assets) {
                const { name, uri } = asset; //filename and uri path
                const mimetype = asset?.mimeType;

                //BASE64 computing
                const file = new FileSystem.File(uri); //load the file
                const base64 = await file.base64();

                files.push({
                    FILENAME: name,
                    UPDATED: true,
                    BASE64: base64,
                    MIME_TYPE: mimetype
                })
            }

            setFilesAttached(prev => [...prev, ...files])

        } catch (error) {
            console.error("Error picking or reading file:", error);
            Alert.alert("Error reading files", error)
        }
    }
```

**Listing 5.4:** File picking snippet.

During the visualization of a post, users can interact with the content by performing actions such as adding likes and browsing uploaded files through a carousel view. Furthermore, the sharing functionality allows users to distribute the post content through external

applications by exploiting the Android Intent mechanism, which enables communication between applications by describing the type of operation and data to be handled. The Android system then identifies the applications compatible with the requested content through their registered Intent filters, as shown in Figure 5.2.

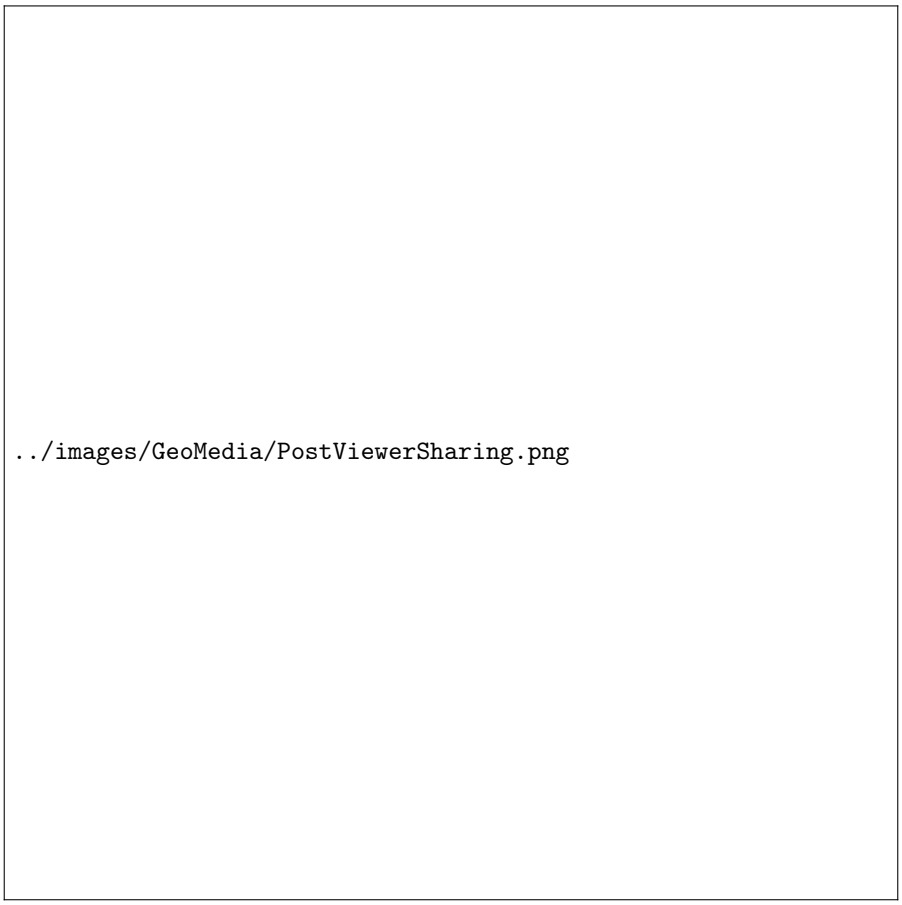


**Figure 5.2:** Post visualization and Android sharing menu

Thus, to upload captured images or selected files to the server, the content is encoded from its binary representation into Base64 format. This encoding provides a standardized method for representing binary data as a textual format, allowing files to be safely transmitted through protocols and systems that handle text-based data. On the application side, the server will decode and store files into the File System of the server.

## 5.2 App flowchart

To provide a comprehensive overview of the application structure and user interactions, a workflow diagram of GeoMedia is presented in the Figure. 5.3

This diagram describes the main operations performed by users and the corresponding processes executed by the application, highlighting the interaction between the user interface, internal components, and external services. The represtation of the application flowchart clarify the sequence of actions and the dependencies between differnet functionalities expecially during app design process.

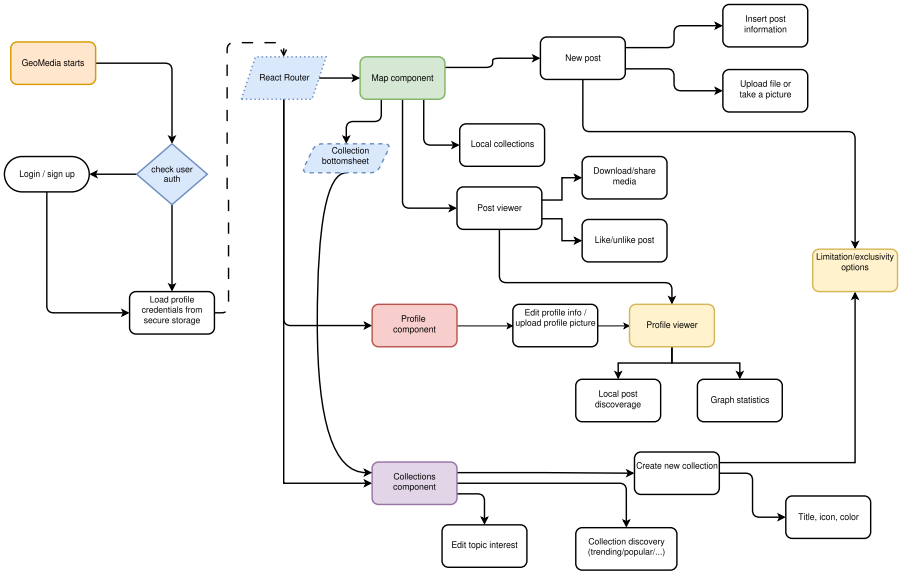


**Figure 5.3:** Application flowchart

## 5.3 Usability

As an Online Social Network, GeoMedia, aims to be used by a potentially large and diverse user base. Consequently, usability represents an important aspect of the system design, expecially for the mobile application, where the limited screen size and touch-based interaction introduce additional constraints. During the development of the mobile app, established usability principles were taken into consideration.

In particular, the interface follows the *thumb-friendly* design principle, according to which frequently performed actions should be positioned within areas of the screen that can be easily reached with the user's thumb when operating the device with one hand. This approach aims to reduce unnecessary hand movements and make common interactions more accessible as shown in Figure 5.4.

However, the current implementation does not account for differences in users' dominant hand. In particular, left-handed users may experience reduced reachability when interacting with controls positioned according to a right-handed usage pattern. To further improve

usability, GeoMedia could provide an option in the settings, for users to specify their dominant hand and dynamically adapt the placement of some buttons. This would allow the interface to better accommodate different interaction patterns and provide a more personalized and accessible user experience.

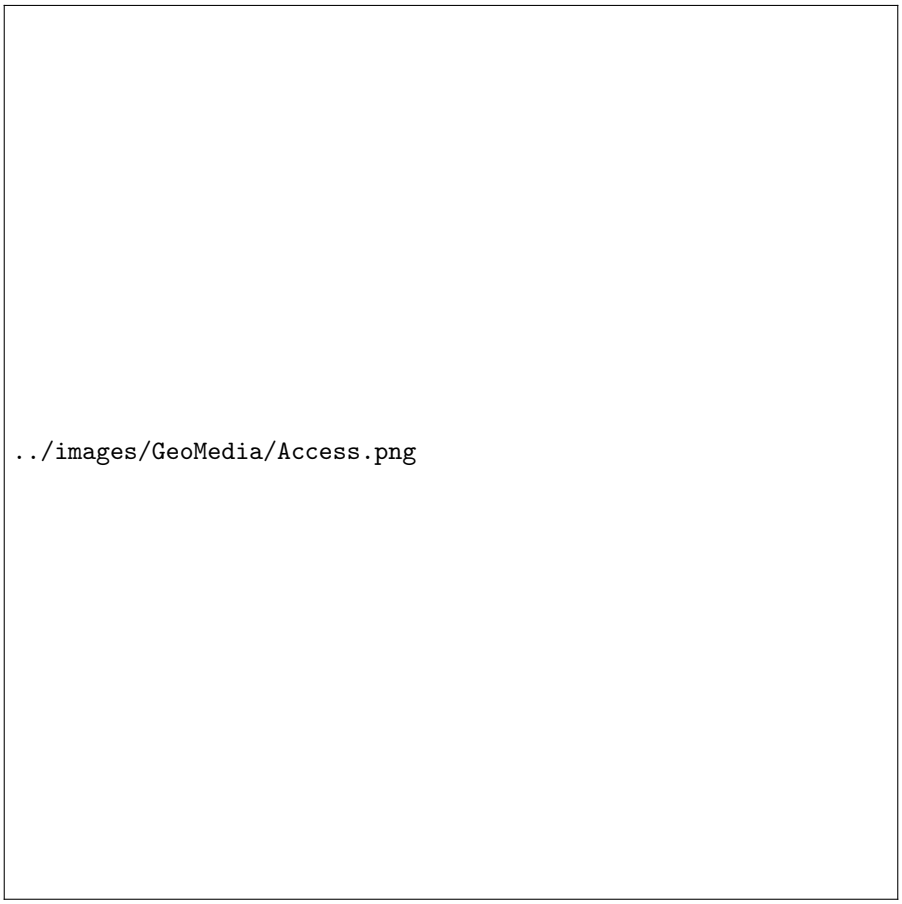


**Figure 5.4:** GeoMedia thumb-rule comparison

Moreover, GeoMedia incorporates gesture-based interactions and haptic feedback to provide a more natural and intuitive interaction experience. Gestures allow users to interact with the application through direct and familiar touch-based actions, while haptic feedback provides immediate physical feedback in response to specific interactions. As an instance, in Figure 3.1, users can reorder posts through a drag-and-drop gesture, providing a seamless and intuitive interaction experience.
Together, these mechanisms aim to improve the perceived responsiveness of the interface and make interactions feel more engaging and consistent.

The design of GeoMedia was also influenced by established social networking applications with which users are likely to be familiar, such as WhatsApp and Instagram. In particular,

the application adopts a bottom navigation menu to provide access to its main sections. Reusing a familiar navigation pattern reduces the need for users to learn a new interaction model and contributes to a more predictable and consistent user experience.

Accessibility was also considered during the implementation of the mobile application. GeoMedia provides support for screen-reader technologies, including Android's TalkBack, by assigning appropriate accessibility roles and descriptive labels to interactive components. This allows assistive technologies to communicate the purpose and state of interface elements to users who rely on them. An example of this implementation is shown in Listing 5.5.

Specifically, accessibility in React Native is implemented through properties of some components, such as `accessibilityRole` and `accessibilityLabel`,providing semantic information to assistive features. For instance, `accessibilityRole` can be used to specify the semantic purpose of an interface element, while `accessibilityLabel` provides a descriptive text that can be announced by screen readers.

This approach is consistent with the accessibility model of React Native, which allows accessibility information to be added directly to components without requiring additional wrapper elements. Perinello and Gaggi's analysis of React Native and Flutter[7], highlights this as an "enhancing" approach to accessibility, in which accessibility properties can be applied directly to existing components; moreover, React Native thorught these properties can also implement featrues such as headings and text abbreviations.

In GeoMedia, these properties are used to provide meaningful descriptions and semantic roles for interactive elements, allowing users relying on assistive technologies such as screen readers to understand and interact with the application.

```
<TouchableOpacity style={[style.buttons.full_screen, style.colors.geomedia_green]}
    onPress={() => { saveInfo() }}
    accessibilityRole="button"
    accessibilityLabel={langselected?.save}
>
    <ThemedText>{langselected.save}</ThemedText>
</TouchableOpacity>
```

**Listing 5.5:** Accessability role in React Native

In the end, GeoMedia mobile application supports both light and dark themes and can automatically follow the device's system-wide appearance settings, thus to adapt user preferences.

In addition, GeoMedia allows user to change map colors among five different map styles:

- **Light**: the default map style for the light theme;
- **Dark**: a dark-themed map style designed for use in darker environments;
- **Blue**: a map style characterised by a night-blue colour palette;
- **Retro**: a map style inspired by traditional cartographic aesthetics;
- **System default**: automatically switches between the light and dark map styles according to the device's system theme.

These options provide users with greater control over the application's visual appearance and allow the map to be adapted to different environmental conditions and individual preferences.

### 5.3.1 SUS Questionaire results

In order to get the qualitative evaluation of GeoMedia's usability, a quantitative assessment was conducted using the *System Usability Scale* (SUS) among 15 different people with different ages and background.
The SUS is a standardized questionnaire originally proposed by Brooke [3] to provide a quick and practical measure of the perceived usability of a system. It consists of ten statements evaluated using a five-point Likert scale, ranging from "Strongly disagree to Strongly agree".

The SUS is a standardized questionnaire originally proposed by researcher John Brooke [3] to provide a quick and practical measure of the perceived usability of a system. Composed by ten statements evaluate using a five-point Likert scale, ranging from "Strongly disagree" to "Strongly agree".
For each statement has been computed the mean value across all participants, thus to provide an overview to analyse the general tendency. Moreover, the standard deviation was calculated to indicate the variability of participants' responses across the questions.

The results of the Questionnaire are reported in Table 5.1

**Table 5.1:** Analysis of SUS questionnaire responses.

| Item | Statement | Mean | SD |
|---|---|---|---|
| 1 | I think that I would like to use this system frequently. | 2.54 | 1.21 |
| 2 | I found the system unnecessarily complex. | 1.54 | 0.78 |
| 3 | I thought the system was easy to use. | 3.61 | 1.21 |
| 4 | I think that I would need assistance to use the system. | 1.60 | 0.70 |
| 5 | I found the various functions in the system were well integrated. | 4.00 | 0.82 |
| 6 | I thought there was too much inconsistency in the system. | 1.70 | 0.82 |
| 7 | I would imagine that most people would learn to use the system very quickly. | 4.10 | 0.74 |
| 8 | I found the system very cumbersome to use. | 1.50 | 0.71 |
| 9 | I felt very confident using the system. | 4.20 | 0.79 |
| 10 | I needed to learn a lot of things before I could get going with the system. | 1.80 | 0.92 |

Reporting both the mean and standard deviation provides a more complete representation of the results by describing both the overall usability level and the consistency of participants' perceptions.

The usability evaluation of GeoMedia produced generally positive results. The highest scores were obtained for users' confidence in operating the system, its learnability and the integration of its functionalities, all receiving scores of `4 or higher out of 5`. These results suggest that users perceived the system as accessible and coherent, with functionalities that could be understood and operated with relative ease.
In contrast, the low score concern the agreement with negative statement concerning system complexity, the need for assistance, inconsistency and cumbersome use; receiving score below of `2 out of 5`. The disagreement with these affirmations indicate that GeoMedia was perceived as relatively simple, consistent and easy to use.

Overall, the first SUS statement, concerning users' intention to use the system frequently, received a comparatively lower value and was therefore investigated further through follow-up analysis involving six of fifteen participants. As an outcome of the interviews, several new or missing features and potential improvements were identified, which are taken into consideration in the conclusion chapter and discussed as possible directions for future development.

## 5.4 Android application

Android is an operating system developed by Google, originally designed for mobile devices such as smartphones and tablets, and later extended to a wider range of platforms, including televisions, vehicles, smartwatches, and other embedded systems. It is based on a modified version of the Linux kernel and was first released in 2008. Since then, it has become the most widely adopted operating system for smartphones worldwide[19].

Android provides a high degree of customization, allowing manufacturers such as Samsung, Motorola o others to modify the user interface and include proprietary applications as part of their devices. Despite these customizations, Android remains an open platform that allows users to install applications from several different stores or, more brutally through APK file, that contain the package and executable code to install the app.

Android application development is freely available, with the main exception for deployment phases to in the official Google stores, where developers need to make one-time payment for a developer license that enable them to upload as many apps they want. Furthermore, Android development is platform-independent, representing a significant advantage over competing platforms such as Apple, where application compilation and deployment require dedicated software tools available only on macOS (Apple Inc operating systems for PC).

### 5.4.1 Development

The development of GeoMedia, since it's a React Native project see different tools cooperating through a well defined sequences in order to develop and build the final application (APK).

The official React Native documentation recommends Expo as the default starting point for most applications, as it provides a production-ready framework with integrated tooling, while still allowing the transition to native workflows when advanced platform-specific functionalities are required.

To develop a React Native application using the Expo framework, the development environment must satisfy a number of software prerequisites:

- **Node.js**: provides the JavaScript runtime environment required to execute development tools and includes the *npm* package manager.
- **npm**: the default package manager for Node.js, used to install and manage project dependencies.
- **Expo**: the framework used to simplify the development, testing, and deployment of

React Native applications. It is installed and managed through *npm* and is typically initialized using the `create-expo-app` utility.

Once the prerequisites have been installed, a new Expo project can be created by executing the command `npx create-expo-app@latest`. The application can then be launched on a target platform using the command `npx expo run:$platform$`, where the platform is `android` or `ios`.
For Android, the application can be executed either on a virtual device (through emulator) or physical if debugging mode is enabled.

The generated project directory contains the application source code, static assets, and several configuration files, including:

- **app.json**: stores the Expo application configuration, including the application name, version, icons, splash screen and platform-specific settings.
- **package.json**: defines the project's metadata, scripts and JavaScript dependencies required by the application.
- **tsconfig.json**: specifies the TypeScript compiler configuration, including compiler options and preferences.

### 5.4.2 Deployment

The deployment phase can be carried out through different distribution channels, starting from the direct sharing of the application package (APK) to publication on dedicated application stores. Each application stores adopts its own policies, technical requirements and review procedures that must be satisfied before an application can be made available to end users.

For the GeoMedia application, the following distribution platforms were considered:

- **F-Droid**: an open-source Android application repository that distributes only Free and Open Source Software (FOSS). Applications published on F-Droid must comply with its transparency and open-source requirements, making it a suitable distribution channel for projects released under open-source licenses.
- **Google Play Store**: the official Android application marketplace maintained by Google. Publishing an application requires the creation of a Google Play Developer

account, compliance with Google's Developer Program Policies and finally the review of the application along automated processess.
Furthermore, for newly registered personal developer accounts, Google requires a closed testing phase involving at least **12 testers for a minimum of 14 consecutive days** before production access can be requested.

- **Obtainium**: an alternative application manager that allows users to install and update applications directly from their official release sources, such as GitHub repositories, without relying on a centralized application store. This approach enables rapid distribution while preserving direct control over application releases.

For testing phase, GeoMedia was distributed through a direct download link to the application's GitHub Releases page, allowing users to download and install the APK independently and checking the previous version as shown in Figure 5.5. Furthermore, with this method, enabled rapid release of new versions while maintaining full control over the deployment process. Moreover, this approach was chosen to simplify the deployment process and facilitate early testing without requiring users to participate in Google Play's closed testing program or to install alternative application stores, such as F-Droid.

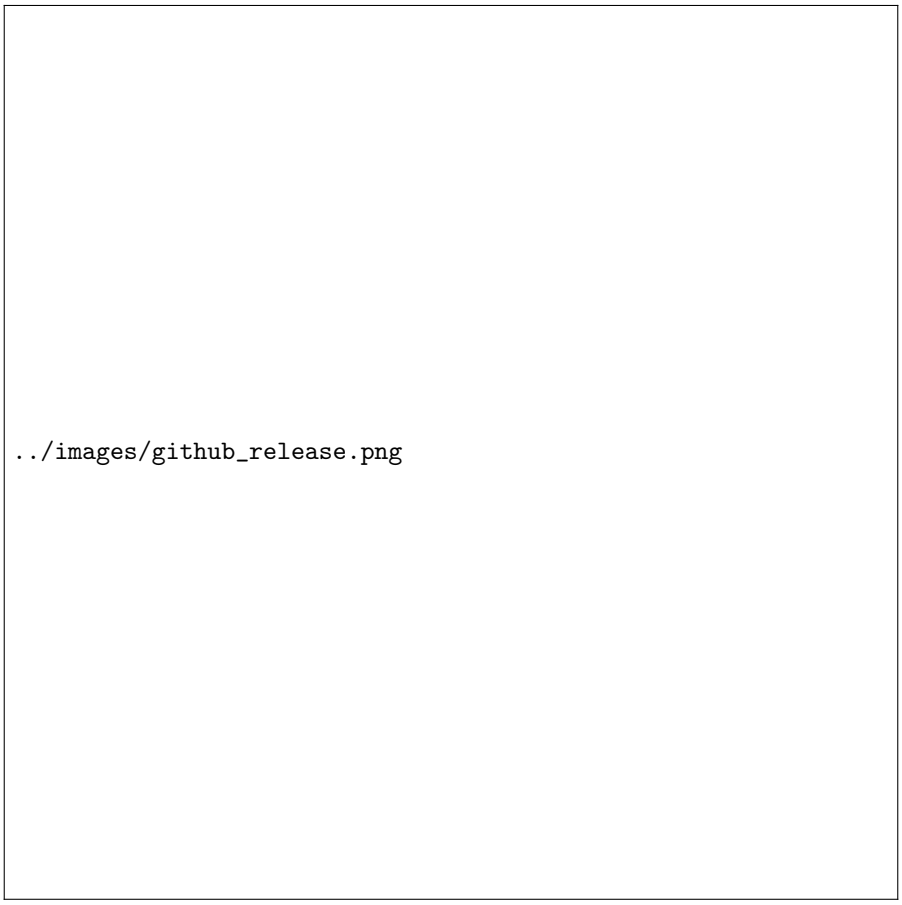


**Figure 5.5:** GitHub release page for GeoMedia

### 5.4.3 APK signing

In order to distribute the application, Android platform, require the APK to be digitally signed before the installation. The signing process consists of generating a cryptographic key pair, using developer's private key to create a digital signature and associate the mark with a certification containing the corresponding public key. During installation and updates, the operating system verifies the signature to ensure the APK's integrity and confirm that it originates from the same signing identity.
This mechanism prevents unauthorized modification and redistribution of the application, as any alteration to the APK invalidates its digital signature. Furthermore, the use of the same signing key across application updates allows Android to establish continuity between different versions and prevents an attacker from installing a modified application as a legitimate update.

The signing process can be performed through Android Studio, which is also used to

compile the application source code thus to build the APK. Developers are enabled to generate the certificate, rather than require it by a trusted Certification Authority. In this way, developers can manage their own singin keys, with each define a distinct signing identity. An example of this process is illustrated in Figure **??**.

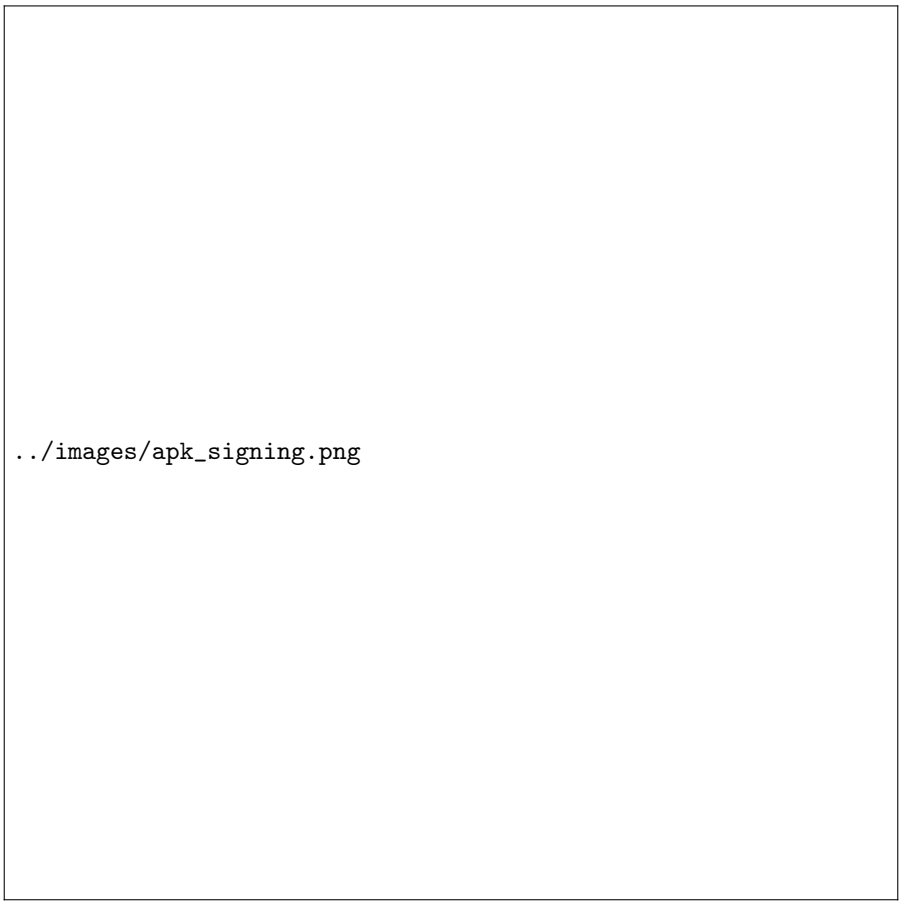


**Figure 5.6:** GitHub release page for GeoMedia

### 5.4.4 Android Manifest

The Android Manifest is a configuration file that defines essential information about an Android application, including its identifier, permissions and required system capabilities. For GeoMedia, the manifest was manually modified (from the React Native generated) to allow clear-text HTTP traffic, enabling users to configure their own server endpoints and adapt the application to different environments.

A snippet of some permissions and the clear traffic usage (disabled by default since Android 8) is shown in Listing

A snippet of some of configurated permissions and the clear-text traffic option is shown

in Listing 5.6. The usesCleartextTraffic attribute was explicitly enabled because Android 9 (API level 28) introduced a default restriction on unencrypted HTTP communication, requiring applications that rely on HTTP connections to opt in explicitly.

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
  <uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION"/>
  <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION"/>
  <uses-permission android:name="android.permission.CAMERA"/>
  <uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE"/>
  <uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>
  <uses-permission android:name="android.permission.VIBRATE"/>
  ...
  <application android:usesCleartextTraffic="true" android:name=".MainApplication"
    ...>
    <meta-data android:name="com.google.android.geo.API_KEY" android:value="..."/>
  </application>
</manifest>
```

**Listing 5.6:** Android Manifest snippet.

## 5.5 Network capability

As designed, the communication among the mobile devices and the server of GeoMedia, is performed through network capabilities and adopting the HTTP protocol (explained in chapter 6.1). However, to perform the sending of respective HTTP packets, the client app do not implements any external interfaces or packages, but a wrapper function for the JavaScript native method `fetch` has been implemented in Listing 5.7.

```
export const doRequest = (api, body = {}, method = "POST") => {
    const nonce = Crypto.randomUUID();
    const timestamp = Date.now().toString();

    let methods_no_body = ["GET", "DELETE"];
    if (methods_no_body.includes(method)) {
        if (body && Object.keys(body).length > 0) {
            const query = new URLSearchParams(body).toString();
            api += "?" + query;
        }
```

```
    }
    // Sign the FINAL path, including query parameters
    const dataToSign = "/" + api + timestamp + nonce;
    const signature = CryptoJS.HmacSHA256(
        dataToSign,
        SHARED_KEY
    ).toString(CryptoJS.enc.Hex);

    const headers = {
        "Content-Type": "application/json",
        "X-Timestamp": timestamp,
        "X-Nonce": nonce,
        "X-Signature": signature
    };

    if (methods_no_body.includes(method)) {
        return fetch(URL + api, {
            method,
            headers: new Headers(headers),
        }).then(res => res.json());
    }

    return fetch(URL + api, {
        method,
        headers: new Headers(headers),
        body: JSON.stringify(body)
    }).then(res => res.json());
};
```

**Listing 5.7:** JavaScript fetch method wrapper.

The wrapper function generates the HMAC signature required to authenticate the request on the server side (illustrated in chapter 6.3.2) and then constructs the HTTP request by setting the appropriate method, headers and the optional body.
Finally, it sends the request to the server and returns the response content parsed in JSON format to the calling procedure which will interprets the data.

# Chapter 6

# Geomedia Backend

The application layer consists of an HTTPS server responsible for implementing the platform's business logic and handling requests from client applications, namely the mobile application. The main role is to process user requests and provide the required functionalities, such as content retrieval, content uploading or authentication, but also implement core operations for GeoMedia such as the restrictions of post visible from user position by implementing the Haversine Formula.3.3.2

Furthermore, this layer acts as virtual bridge between the mobile application and the data layer, as previously introduced in the Chapter 4.
However, as previously mentioned, this design follows a monolithic architecture, in which the HTTPS server operates as a single, self-contained unit responsible for handling all application functionalities and requests. While this approach is well suited for relatively simple applications with limited traffic and modest computational requirements, it may become a limiting factor as the platform grows. To improve performance, scalability, maintainability, and extensibility, the application layer can be decomposed into a set of independent microservices. In a microservices architecture, each service is responsible for a specific functionality and can be developed, deployed and scaled independently. This modular approach enhances fault isolation, simplifies maintenance and allows the system to adapt more efficiently to increasing workloads and evolving functional requirements.

To implement the backend of the GeoMedia platform, the Python programming language was selected due to its rich ecosystem of libraries, readability and rapid development capabilities. In particular, the FastAPI framework was adopted as it offers high performance, native support for asynchronous programming and automatic API documentation (known

as Swagger).

## 6.1 REST Server

A REST server is a software application that implements the HTTP/HTTPS protocol and continuously listens for incoming client requests. It exposes one or more RESTful endpoints through which clients can interact with the services offered by the application.

More specifically, the server listens for HTTP or HTTPS requests on a configurable network port. A network port is a logical communication endpoint that allows the operating system to route incoming network traffic to the application.

For each server or simply, computer, the port numbers range start from `0` to `65'535` and are conventionally divided into three categories:

- **Well-known ports (0–1023)**: Reserved for standard services and protocols, such as SMTP (port 25, mail protocol), HTTP (port 80), HTTPS (port 443) or SSH (port 22, remote connection).
- **Registered ports (1024–49'151)**: Assigned to specific applications and services, although they may also be used by custom applications.
- **Dynamic or private ports (49'152–65'535)**: Typically allocated dynamically by the operating system for temporary client connections or available for private application use.

By binding a server process to a specific port, operating system forwards all requests addressed to that port to the corresponding application that will handle the HTTP request and return a response (respecting the client-server model).

The HTTP is an application-layer protocol that defines the format and exchange of messages between clients and servers on the Internet. Rather than providing communication mechanisms itself, HTTP relies on the services offered by the underlying layers of the *TCP/IP* networking stack, where each layer is responsible for a specific set of operations. the TCP/IP model division promote modularity, encapsulation, robustness and separation of responsibilities, thus to allow each layer to evolve independently without affecting the others.

The TCP/IP model is represented by four layers:

- **Application layer**: provides network services directly to user applications. Protocols such as HTTP, HTTPS, FTP, SMTP, DNS operate at this level, defining how data is

formatted and exchanged between communicating applications.
*GeoMedia's server use the HTTPS protocol of this stack.*

- **Transport layer**: ensures end-to-end communication between processes running on different hosts. The two principal transport protocols are TCP (Transmission Control Protocol), which offers reliable, ordered and error-checked delivery of data, and **UDP (User Datagram Protocol)**, which provides a lightweight, connectionless communication service with lower latency but without delivery guarantees.
*GeoMedia's server adopt the TCP protocol of this stack.*

- **Internet layer**: responsible for logical addressing and packet routing across interconnected networks. The **Internet Protocol (IP)** determines how packets are addressed and forwarded from the source host to the destination, regardless of the physical networks traversed. There are two principal versions of the Internet Protocol (IP): IPv4 and IPv6. While both protocols provide logical addressing and packet routing across interconnected networks, IPv6 introduces several improvements over its predecessor.
*GeoMedia backend is deployed using IPv4 networking.*

- **Link layer**: handles communication over the local physical network. It is responsible for framing, physical addressing (e.g., MAC addresses), error detection on the local link, and transmitting data through the underlying network technology, such as Ethernet or Wi-Fi.

When an HTTP request is generated by a client, it is encapsulated as it traverses the TCP/IP stack, with each layer adding its own protocol-specific information before transmission. Upon reaching the destination, the reverse process, known as *decapsulation*, removes these protocol headers and reconstructs the original HTTP message, which is then delivered to the server application.

An HTTP message is composed of three main components:

- **start line**: which specifies the HTTP method, like `GET` conventionally used for request a resource, `POST` conventionally used to upload data, `DELETE` to request the deletion of a resource and few others. Together with the `path` (everything after the base url of server, such as query string or paths) indicate the the conventional operation to apply on the resource.

- **header**: a small portion of packet that contain metadata describing the request or response, such as the `Content-Type` which indicates the format of the transmitted data

(e.g., `application/json`, `text/html`), authentication information, caching directives and others parameters.

- **body**: only with methods such as `POST` or `PUT`, contains the actual payload exchanged between client and server, such as JSON objects, uploaded files or text.

### 6.1.1 Business logic

Conceptually, the server defines the endpoints and determines how requests are handled, retrieving or storing data through the data layer by executing the respective query commands or managing the file system. For example, hypermedia are stored in the server's file system, since storing them in the database as encoded text could result in worse performance and increased storage requirements. Conversely, to read the hypermedia referenced to a post whenever opened, the server must encode the media in Base64 format thus to encapsulated it in the HTTP response packet as shown in Listing 6.1, then the mobile app will provide to render to user.

```
async def hpmedia_read_folder(postid: int):
    query = f"EXEC HPMEDIA_GETFILES @POSTID={postid}" # SQL hypermedia reference
    files = SQL_MANAGER.select_query(query)
    for f in files:
        try:
            with open(f.get("FILEPATH"), "rb") as file:
                f["BASE64"] = base64.b64encode(file.read()).decode()
        except Exception:
            f["BASE64"] = None
    return files
```

**Listing 6.1:** Hypermedia retrieval implementation in Pyhton.

However, for GeoMedia application, the business logic is implemented mainly in the data layer, leaving the server applies the final filters and logic; as previously shown in the implementation of the Haversine formula 3.3.2, used to compute the distance between the current location of the user and the posts. The same function is also leveraged by the *Local collection* feature as shown in Listing 6.2.

```
async def collections_get_local(uid,curr_lat,curr_lon):
    query = """EXEC POST_GETALL_ALLOWED @UID=%s"""
    posts = SQL_MANAGER.select_query(query, (uid))
```

```
    results = []
    ## filter by allowed by visibility
    for pp in posts:
        if check_post_area(
            pp["VISIBILITY_AREA_KM"], # post range
            pp["LATITUDE"], # post lat
            pp["LONGITUDE"], #post lon
            curr_lat, #user curr lat
            curr_lon #user curr lon
        ):
            results.append(pp)

    collection_rank = dict()
    for p in results:
        c = p["COLLECTION_ID"]
        if(c not in collection_rank):
            collection_rank[c] = 1
        else:
            collection_rank[c] += 1

    collection_ids = [
        k for k, v in sorted(collection_rank.items(),key=lambda item:
    item[1],reverse=True)[:10]
    ]
    id_collections = ", ".join(map(str, collection_ids))

    query = f"SELECT * FROM COLLECTIONS WHERE ID IN ({id_collections})"
    return SQL_MANAGER.select_query(query)
```

**Listing 6.2:** Python implementation for algorithm used to compute the local collection visible from current location.

In this implementation, the local collections are retrieved from the post that user is allowed to see from its current location. Then, the collection are ranked based on the number of visible posts belonging to each collection, and finally the ten most used collections are selected, retrieved from the database and returned to the calling function that will respond to the HTTP request.

## 6.2 Python FastAPI

FastAPI is a web framework implemented in the Python backend to instantiate a HTTP-based service APIs. It uses Pydantic as validation library, moreover it provides a declarative way to specify the structure and types of data incoming request (e.g., HTTP bodies) and outgoing responses.

The framework simplifies the definition of RESTful endpoints, allowing developers to associate a Python function directly with the HTTP method and resource path thought the use of decorators. The path may include both *path parameter* which are embedded within the URL and *query parameter* that are appended to the request URL as key-value pairs. FastAPI automatically validates these parameters according to the declared Python types and makes them available to corresponding request handler.

In Listing 6.3 is shown the implementation of the endpoint responsible for retrieving the posts visible from the current location of a target profile (function visible in the right image of Figure 3.5). The process is associated with `GET` method and defines query parameters (that are not part of the url) which are passed to the respective function associated to retrieve the corresponding information from the data layer (as shown in Listing 6.4) and returns the result as an HTTP response.

```
1
2 @app.get("/profile/show_allowed_post",tags=["USERS"])
3 async def profile_show_allowed_post(uid: int, profile_id: int, curr_lat: float,
      curr_lon: float):
4     return await
      geomedia_helper.profile_show_allowed_post(uid,profile_id,curr_lat,curr_lon)
```

**Listing 6.3:** HTTP Server Endpoint creation.

```
1
2 async def profile_show_allowed_post(uid, profile_id, curr_lat, curr_lon):
3     ## prepared statement to avoid SQL Injection
4     query = """
5     EXEC PROFILE_GET_ALLOWEDPOST
6         @UID=%s,
7         @PROFILE_ID=%s
8     """
9     ## execute the query
```

```
    posts = SQL_MANAGER.select_query(query, (uid, profile_id))

    results = []

    # Filter by visibility
    for pp in posts:
        if check_post_area(
            pp["VISIBILITY_AREA_KM"],
            pp["LATITUDE"],
            pp["LONGITUDE"],
            curr_lat,
            curr_lon,
        ):
            results.append(pp)

    return results
```

**Listing 6.4:** Implementation of post retrieval of a target user visible from current location.

### 6.2.1 Documentation

FastAPI automatically generates an API specification in *OpenAPI* format, represented as a JSON document. The framework also exposes interactive documentation endpoints, provided via textitSwagger UI (typically available at `/docs`), which provides a comprehensive description of every available endpoint. Moreover, consent to group by endpoints in order to create some classification through tags, in GeoMedia as example, `AUTH` for authentication operations, `PROFILE` for profile functionality or `COLLECTIONS` for collection related endpoints. For each endpoint, the documentation specifies the supported HTTP method, expected request parameters, request body schema, authentication requirements and the format and status codes of the corresponding responses.
Since the documentation is generated directly from the application source code, it remains synchronized with the API implementation throughout the development process.

An example of GeoMedia backend API documentation is shown in Figure 6.1

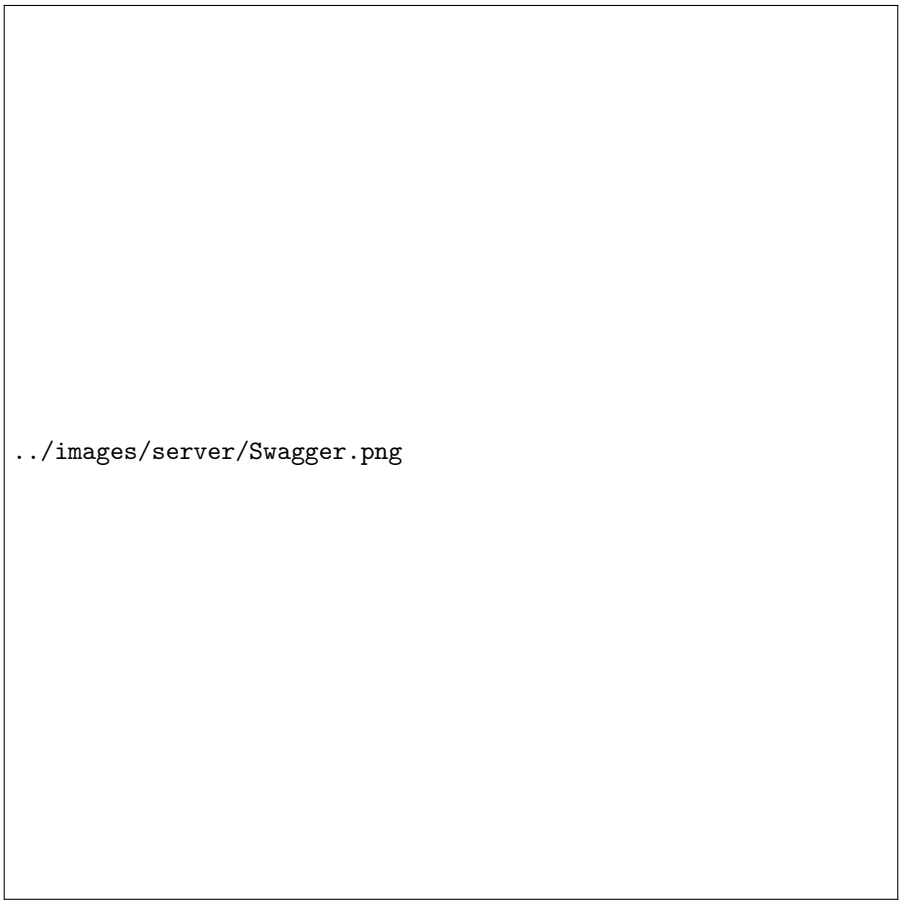


**Figure 6.1:** GeoMedia API documentation example

### 6.2.2 NodeJS HTTP Comparison

The first version of the GeoMedia backend adopted HTTP native module of NodeJS for the application layer. Although the high performance and event-driven, asynchronous execution model, this solution became increasingly difficult to maintain as the number of API endpoints and application features grew. In specific, the absence of a well-structured framework reduced modularity and created a codebase that was progressively harder to extend.

Compared to FastAPI, Node.JS ecosystem commonly relies on Express framework, which provides a lightweight and flexible approach for creating HTTP servers, offering routing mechanisms and a middleware-based architecture, where additional functionalities can be integrated through a chain of middleware components.

Therefore, while Express offers greater flexibility and a mature ecosystem, FastAPI provides additional built-in functionalities that simplify the development and maintenance

of scalable RESTful backends. For the GeoMedia platform, FastAPI was selected because provide integrated validation, documentation and structured development model better support the long-term evolution of the application layer.

## 6.3 Cybersecurity & Privacy

In order to ensure the privacy, integrity, availability and confidentiality of data, GeoMedia, must adopt some mitigation techniques to prevent cyber-attacks especially because the platform processes user accounts, user-generated content and location-related information.

The security concerns implemented follows a defends in every layer of the application architecture. The complementary adoption of different mechanism consists in secure communication protocols, authentication and authorization and preventing injection attacks.

Moreover, user privacy is a core principle of GeoMedia. As an open-source application, its codebase allows users to analyse and validate the application's behaviour, as well as inspect how user data is collected, processed, and managed.

### 6.3.1 Account Verification

To ensure the authenticity of account, users, during the registration provide an email address that will be associated to their account. On registration, the backend generates a One-Time Password (OTP) that is sent to the registered email address and must be entered within a limited period of time (imposed to 1 hour). Once successfully verified, the account is verified and user is allowed to utilize the platform.

The use of a time-limited OTP provides an additional security layer compared to simply accepting an email address supplied during registration. In particular, the verification mechanism helps reduce the creation of accounts using invalid or inaccessible email addresses. The risk of brute-force attacks against the OTP verification procedure, is mitigated by limiting the number of verification attempts up to five, as implemented in Listing 6.5. Once the maximum number of attempts has been reached, the account results blocked and user must contact the GeoMedia back-office or proceed with a new account.

```
1
2 PROCEDURE [dbo].[AUTH_CHECK_OTP] @JSON NVARCHAR(MAX)
3 AS BEGIN
4 IF EXISTS(
5         SELECT * FROM USERS WHERE USERNAME = JSON_VALUE(@JSON, '$.USERNAME') AND
```

```
    OTPCODE = CAST(JSON_VALUE(@JSON, '$.OTP') AS INT)
        AND DATEDIFF(hour, OTPDATE, GETDATE())<= 1 --OTP generated within an hour
        AND BLOCKED=0 --TO PREVENT OTP BRUTEFORCE
    )
    BEGIN
        --VERIFY THE PROFILE
    ELSE
    BEGIN
          UPDATE USERS
            SET WRONG_OTP_ATTEMPT=WRONG_OTP_ATTEMPT+1,
            BLOCKED = IIF(WRONG_OTP_ATTEMPT>4,1,0) -- UP TO 5 ATTEMPTS
            WHERE USERNAME = JSON_VALUE(@JSON, '$.USERNAME')

    END
```

**Listing 6.5:** OTP verification processed by data layer, and brute-force mitigation.

The OTP generation and verification operations are performed in the Data Layer, in this case the server operates just as intermediary for the HTTP requests.

### 6.3.2 HTTPs and packet confidentiality

All communication between the client and GeoMedia's backend is performed through HTTPS. HTTPS relies on TLS (Transport Layer Security) to provide confidentiality and integrity for data exchanged between the client and server. This prevents an attacker positioned between the client and the server from trivially observing or modifying authentication requests.

Finally, the communication process between clients and server, can be summarized as:

1. The client establishes a connection with the server and requests a secure TLS connection.
2. The server provides a digital certificate containing its public key and information about its identity.
3. The client verifies the certificate against trusted Certificate Authorities (CAs).
4. During the TLS handshake, the client and server securely establish shared session keys using public-key cryptography.

5. Once the handshake is completed, the HTTP traffic is encrypted and authenticated using symmetric cryptography and the established session keys.

As additional security mechanism, GeoMedia, to verify the integrity and authenticity of messages exchanged between trusted components adopt HMAC (Hash-based Message Authentication Code) mechanism.

HMAC combines a cryptographic hash function with a secret key. Despite simple hash, that only provides a digest of the input, HMAC requires knowledge of a shared secret key. The receiver that possess the same secret can verify whether a message was generated by the authorized GeoMedia official application or control if a third part has modified the packet.

Consequently, a recipient possessing the same secret can verify whether a message was generated by an authorized party and whether its content has been modified.

Conceptually, an HMAC can be represented as:

$$\text{HMAC}(K, m)$$

where $K$ represents the secret key and $m$ represents the message.

In particular, GeoMedia generates an HMAC for each request using the URL path, query parameters, a nonce plus a timestamp as the message. The HMAC is computed using a shared secret key known by both the client and the server. The timestamp and nonce are subsequently validated by the server to control the validity of the request and mitigate replay attacks. In this way, a previously captured request cannot be reused indefinitely, as the server can reject requests containing an expired timestamp or an already-used nonce.

### 6.3.3 Password handling

Another cryptographic technique adopted by GeoMedia is the MD5 hash function. Unlike encryption, hashing is a one-way process that transforms an input, such as a password, into a fixed-length digest. In GeoMedia, the MD5 hash is computed by the client mobile application before the password is transmitted to the backend, which store the digest on the server through the data layer.

### 6.3.4 Password handling

Another encryption techniques, adopted by GeoMedia, is the MD5 hash function, which it's an injective function that represent one-way only to encrypt a text such as password, in

order that neither seeing the digest (result of the hash) can be reverse to obtain original password.

The MD5 is applied from the client mobile application and stored by server in a respective table of data layer.

### 6.3.5 Protection Against SQL Injection

SQL Injection is a major application-layer vulnerability in systems that interact with relational databases. The vulnerability occurs when user-controlled input is incorporated directly into SQL statements, allowing an attacker to manipulate the intended query.

For example, constructing a query through string concatenation is unsafe:

```
SELECT * FROM users WHERE email = ' + userEmail + '
```

An attacker could provide specially crafted input that changes the semantic meaning of the SQL query.

To mitigate this vulnerability, GeoMedia, through the usage of prepared statements ensure the correct execution of SQL query and execution of stored procedure through the DBMS (Data Layer).

Listing 6.6 shows the use of a prepared statement for the execution of a stored procedure. The use of parameterized queries ensures that input provided is treated as data rather than executable SQL code. This prevents a malicious input, such as the request in Listing 6.7, from altering the intended query and retrieving unauthorized information, such as the data of all users stored within the application.

```python
async def post_delete(postid: int, password: str):
    writeLog(f"Requested deletion for: {postid} = with pas[10]{password[:10]}")
    # Perpared statement to avoid SQL INJECTION
    query = """
    EXEC POST_DELETE @POSTID=%s, @PASSWORD=%s
    """
    return SQL_MANAGER.insert_query(query, (postid, password))
```

**Listing 6.6:** Perpared statement example to mitigate SQL Injection.

The value supplied by the user is subsequently bound to the parameter by the database driver. Consequently, the database treats the supplied value as data rather than as executable SQL syntax.

```
import requests
requests.delete("https://geomediasrv.duckdns.org:9911/post/1",
    headers={...},
    params={"password": "x'; SELECT * FROM USERS';--"}
)
```

**Listing 6.7:** Request example for SQL Injection.

Another injection attack that targets website and web-application is the cors-site scripting, where an attack submit malicious HTML or JavaScript as part of a content (such as post, profile field, comments etc.). Then, the rendering of the content without appropriate sanitation or encoding, makes the user's device (such as the browser) execute the malicious script.

However, this attack cannot be performed against GeoMedia as the React Native mobile application does not use a WebView component. In fact, the app renders user interface using native platform component translated during building process, while JavaScript code is bundled in binary form. Nevertheless, this attack may be feasible in others mobile framework such as IonicFramework, which was used in the first version of GeoMedia, as parts of the application are executed within a WebView.

### 6.3.6 Content Moderation and Abuse Prevention

As an Online Social Network, GeoMedia allows users to create and distribute content. Consequently, cybersecurity is not limited to protecting the technical infrastructure but also includes mechanisms for mitigating platform abuse.

Users are able to report a posts through an opportune form shown in Figure 6.2, then to flag potentially harmful posts and add a mandatory motivation. When a post receives more than three reports from distinct users, it is automatically hidden from the map and cannot be discovered since the moderation process has evaluate the content attainability.

The moderation process can involve either manual review or automated classification, in latter case, Large Language Models (LLMs) can be used to analyse potentially harmful comments, while computer vision models or multimodal LLMs can be adopted to identify inappropriate images. Because automated classification systems may produce false positives and false negatives, LLM-based moderation should not necessarily be considered an infallible security mechanism. Particularly sensitive decisions should provide an escalation path to human review whenever there are isn't sufficiently certain.

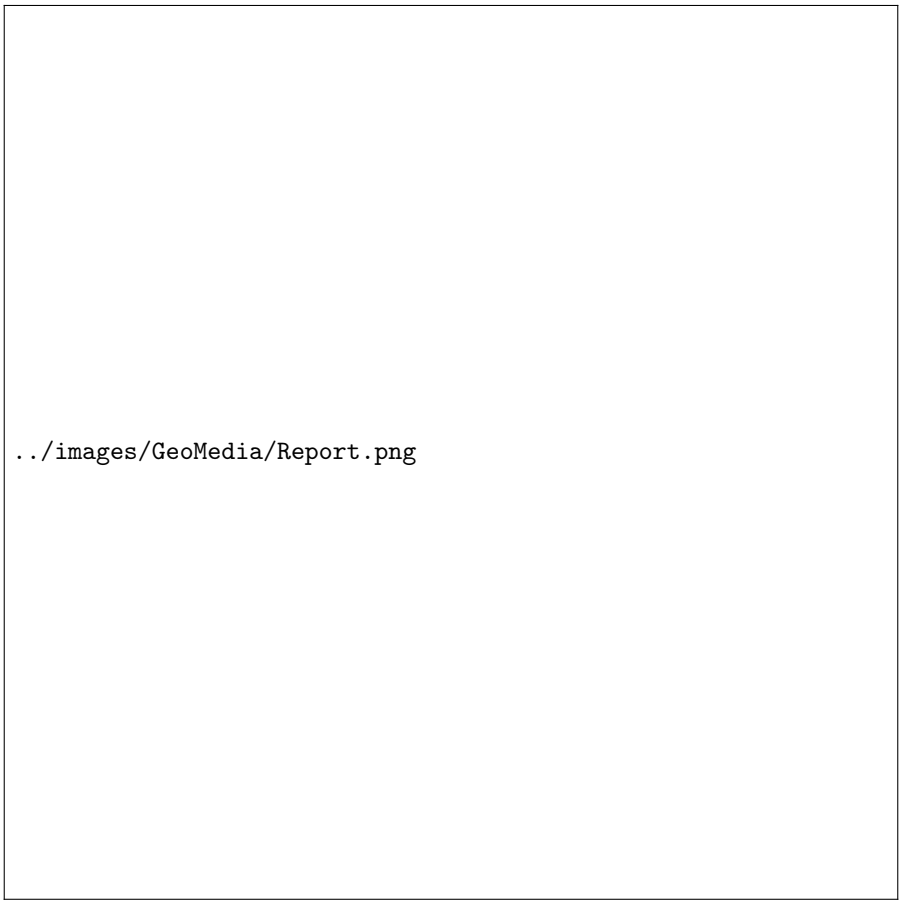


**Figure 6.2:** Post reporting.

The use of reports from distinct users is important because it reduces the effectiveness of repeatedly reporting the same content from a single account. Nevertheless, the reporting mechanism itself represents a potential abuse vector and should therefore be protected through mechanisms such as rate limiting and detection of suspicious reporting behavior.

### 6.3.7 GDPR and Privacy Considerations

Since GeoMedia processes personal data and is operated within the European context, security mechanisms must also be considered in relation to the General Data Protection Regulation (GDPR).

As Cybersecurity focuses primarily on protecting systems and informations against unauthorized access, alteration and destruction; Privacy, concerns how personal data is collected, processed, stored and shared.

GeoMedia apply principles such as data minimization, collecting only necessary information and actuate mechanism such as authentication and encryption thus to respect the

confidentiality.

Authentication-related information, security logs and other potentially sensitive data should also be protected against unauthorized access.

### 6.3.8 Security Summary

The main security mechanisms adopted by GeoMedia can be summarized in the table 6.1.

| *Mechanism* | **Description** |
|---|---|
| **Account verification (OTP)** | During the registration phase, the system verifies that the provided email address belongs to the user registering the account. |
| **HTTPS/TLS** | Protects data exchanged between clients and backend services, applying encryption. |
| **HMAC** | Used to verify the authenticity and integrity of messages and requests, and combined with nonces or timestamps, to mitigate replay and masquerade attacks. |
| **HASH (MD5)** | Used to hash passwords into a fixed-length representation before storage. |
| **Prepared Statement** | Implemented to mitigate SQL injection attacks by preventing user input from being interpreted as part of SQL queries. |
| **Content moderation** | Designed to support automated moderation using LLMs and human review to identify and manage potentially harmful content. |

**Table 6.1:** Cybersecurity mechanism implemented.

Overall, GeoMedia adopts a layered security model in which preventive, detective, and corrective controls operate together. This approach allows the platform to address both traditional application-level threats, such as SQL Injection and XSS, and threats specific to an Online Social Network, including account abuse, malicious user-generated content, automated reporting, and unauthorized access to protected resources.

# Chapter 7

# Data Layer

The Data Layer, also referred as Data Access Layer (DAL) represent the access and handling to data from one or more source such as different relational database, NoSQL database, file systems or others persistent storage.

Data Layer commonly used to centralize data access logic, reducing redundances and providing a consistent interface for retrieving or storing data.

For GeoMedia, the data layer is implemented using a Relational Database Management System (RDBMS), that represent the most common and well-established approach for the structured management, storage and retrieval of data. The RDBMS organizes information into logically related tables composed of rows and columns, while providing mechanisms for defining relationships, enforcing data integrity and efficiently executing complex queries and update operations through Structured Query Language (SQL) instructions.

For the implementation of GeoMedia, Microsoft SQL Server (MSSQL) has been selected as the underlying relational database management system due to its robustness, scalability, and comprehensive support for enterprise-level data management. In addition to standard relational database capabilities, SQL Server provides advanced mechanisms for transaction management, data indexing and performance optimization.

MSSQL is freely available through the Express edition, which introduces some limitations in terms of storage (up to 10 GB per database), performance, and administration tooling. However, for the scope of this project, these limitations are considered acceptable. If the platform needs to scale in the future, the GeoMedia database can either be split across multiple instances, as discussed in the microservices section (Figure 4.2), or the system can be migrated to the Standard edition by acquiring the appropriate license.

## 7.1 Database

The database designed for the application follows some well known concepts such as

The database schema designed for the GeoMedia framework follows established relational database design principles, with particular attention to normalization, data integrity, and the appropriate definition of relationships between entities. Specifically, the main principles adopted during the design are:

- **Normalization**: the database schema is structured to minimize data redundancy and prevent insertion, update, and deletion anomalies, while maintaining clear relationships between entities.
- **Referential integrity**: primary and foreign key constraints are used to ensure that relationships between related entities remain consistent and that references to non-existent records are avoided.
- **ACID properties**: database transactions follow the principles of Atomicity, Consistency, Isolation, and Durability, ensuring that operations are executed reliably and that the database remains in a valid state.
- **Data integrity and consistency**: constraints such as `NOT NULL`, `UNIQUE`, and `CHECK` are employed where appropriate to prevent invalid or inconsistent data from being stored.

Moreover, the database schema was designed to facilitate the extensibility of the system by allowing additional entities and relationships to be introduced without requiring substantial modifications to the existing structure.

The Entity–Relationship (ER) diagram in Figure 7.1 provides an overview of the main entities and their relationships within the GeoMedia database. For readability, entity attributes are omitted from the diagram, as several entities contain a considerable number of attributes.

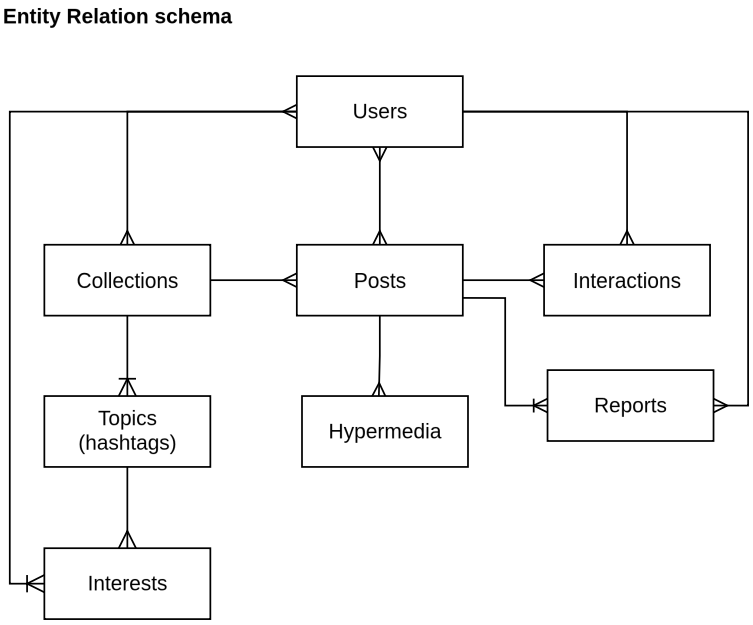


**Figure 7.1:** GeoMedia Entity-Relationship diagram

The entities individuated are:

- **Users**: which stores respective user informations and login credentials such as password and email. Moreover, create a reference to posts, reactions and eventual post reports.
- **Posts**: storing post information and then representing the core entity for integration and references.
- **Collections**: that works as containers for posts, dictating logic properties on posts and allowing special features (e.g. sequential posts).
- **Hypermedia**: operates as register with path of hypermedia stored by server layer in the file system.
- **Interactions**: used to store the user interaction with posts (actually just "like" reactions) and marks the view, used both for view counter and managing post sequentiality features. The table is also a starting point for comment extension.
- **Reports**: store the report raised by users referenced to posts and then enable the moderation process of the platform.
- **Topics**: works as a container of interests that will be seen by GeoMedia application in form of hashtags. Moreover, these can be associated both to collections than to the user.

- **Interests**: that works as register to link users to their topic saved.

In order to ensure the separation of GeoMedia's data from other databases hosted on the same database server, a dedicated database is created for the application using the `CREATE DATABASE GEOMEDIA` instruction. The database contains all the tables required to store the application's data, together with the corresponding stored procedures and functions. Through stored procedures (precompiled SQL routines), data can be manipulated through more complex operations, while business logic can be encapsulated within the database. This approach improves the maintainability and extensibility of the system, while potentially improving performance by reducing the amount of data processing required at the application level. Finally, the stored procedures provide the results whenever requested by the server, as an example shown in Listing 7.1 which deals with storing the topic of interest (in term of hashtags) of a users; specifically, the procedure execute a merge query operation, that basically performs an `insert` (creation) whenever the record do not exists, otherwise process an update by matching with key attributes.

```
CREATE PROCEDURE [dbo].[PROFILE_INTEREST_MERGE]
@UID INT, @JSON NVARCHAR(MAX)
AS
BEGIN

    DECLARE @TABLE TABLE (
        VAL VARCHAR(200),
        ENTITY VARCHAR(200)
    );

    INSERT INTO @TABLE(VAL, ENTITY)
    SELECT value, entity
    FROM OPENJSON(@JSON, '$')WITH(
        value VARCHAR(200),
        entity VARCHAR(200)
    );

    DELETE FROM INTERESTS
    WHERE [USER_ID] = @UID
    AND NOT EXISTS (
        SELECT * FROM @TABLE T
        INNER JOIN HASHTAGS HT ON HT.TITLE = T.VAL AND T.ENTITY = 'HASHTAG'
        WHERE HT.ID = EXTERNAL_REF AND ENTITY = T.ENTITY
    )
    --- CREATE HASHTAG IF NOT EXISTS
```

```
    INSERT INTO HASHTAGS (TITLE, DC, UC)
    SELECT T.VAL, GETDATE(), @UID
    FROM @TABLE T
    WHERE T.ENTITY = 'HASHTAG' AND NOT EXISTS(SELECT * FROM HASHTAGS WHERE TITLE = T.VAL )
    -- INSERT HASHTAG ON INTERESTS
    INSERT INTO INTERESTS ([USER_ID], EXTERNAL_REF,ENTITY)
    -- SELECT @UID, ISNULL(HT.ID,COLLETION.ID--> TO DO WITH CASE WHEN), T.ENTITY
    SELECT @UID, HT.ID, T.ENTITY
    FROM @TABLE T INNER JOIN HASHTAGS HT ON HT.TITLE = T.VAL
    WHERE NOT EXISTS (
        SELECT * FROM INTERESTS WHERE ENTITY = T.ENTITY
        AND EXTERNAL_REF = HT.ID
        AND [USER_ID] = @UID
    )

    EXEC PROFILE_INTEREST_GET @UID = @UID --execute another stored procedure that returns
     all the interest
END;
```

**Listing 7.1:** SQL Stored Procedure with merge operation.

Instead for example of database structure, the SQL definition of the *Posts* table is shown in Listing 7.2.

```

CREATE TABLE GEOMEDIA.dbo.POSTS (
  ID int IDENTITY(1, 1) NOT NULL,
  AUTHOR_ID int NOT NULL, -- author of post
  TITLE varchar(50) NOT NULL, --title of the post
  COMMENT varchar(MAX) NULL,
  DC datetime NULL,
  DM datetime DEFAULT getdate() NULL,
  LATITUDE float NULL, --latitude coordinate
  LONGITUDE float NULL, --longitude coordinate
  ALTITUDE float NULL, -- altitude for future developments
  VISIBILITY_AREA_KM float, -- visibility range
  EXCL_DATE_START datetime NULL, --exclusivity time start
  EXCL_DATE_END datetime NULL, --exclusivity time end
  RECURRENT int NULL, -- recurrency (0 = no reccurent, 1 = monthly, 2 = yearly)
  COLLECTION_ID int,
  VIEWERS nvarchar(MAX) NULL, -- exclusivity viewers
```

```
18   CONSTRAINT PK__POSTS__3214EC27ECDD25A2 PRIMARY KEY (ID),
19     CONSTRAINT FK__POSTS__AUTHOR_ID__4F7CD00D FOREIGN KEY (AUTHOR_ID) REFERENCES
       GEOMEDIA.dbo.USERS(ID);
20     CONSTRAINT FK__POSTS__AUTHOR_ID__7C4F7684 FOREIGN KEY (AUTHOR_ID) REFERENCES
       GEOMEDIA.dbo.USERS(ID);
21     CONSTRAINT FK__POSTS__COLLECTIO__7D439ABD FOREIGN KEY (COLLECTION_ID)
       REFERENCES GEOMEDIA.dbo.COLLECTIONS(ID);
22 );
```

**Listing 7.2:** SQL definition of the Posts table.

## 7.2 Sequential post implementation

The algorithm responsible for managing post sequentiality is implemented in the stored procedure `POST_GET_MAP`. The procedure is responsible for retrieving the posts available to the requesting user, applying the visibility and temporal constraints defined by the application and finally determining the appropriate order of sequential posts. Meanwhile, the radius-based area filter is performed by the backend, as described in Section 3.3.2, on the dataset returned by the stored procedure.

In particular, the procedure handles sequential posts by identifying the first post in the defined sequence that has not yet been viewed by the requesting user, allowing the framework to progressively expose posts according the sequence defined in the respective collections selected by user.

The sequentiality logic is implemented through a CTE (Common Table Expression), that extracts the ordering information associated with collections from `SEQUENTIALS` field stored in JSON format, then applying the SQL Server built-in function `OPENJSON` parse the structure in form of table with corresponding `order_id` and `post_id` values.

The orders values are stored temporarily in the `@SEQQ` table variable, subsequently used to associate each posts with its position within the sequence. INstead, a second CTE `NEXTPOST` identifies the first post in the sequence that has not yet been viewed by the requesting users. This is achieved by comparing the posts associated with the collections against the records stored in the `INTERACTIONS` table, where the value `VIEW` in field `SCOPE` marks the post as seen by the user.

The results are ordered according to the sequential `order_id`, ensuring that the next available

post is selected according to the sequence defined by collection owner. Finally, the procedure combines the next post to be displayed of respective collections with sequentiality enabled, and posts that are not associated with a sequence. Additional filters are applied to respect account visibility (both post than collection), temporal restrictions and respective recurrence rules.

The implementation of this logic is shown in Listing 7.3.

```
CREATE PROCEDURE [dbo].[POST_GET_MAP]
@UID INT, @COLLECTIONS_CHOSEN NVARCHAR(MAX)= NULL
AS
BEGIN
    DECLARE @SEQQ TABLE (order_id int,post_id int);

    ;WITH SEQ AS(
        SELECT j.order_id, j.post_id
        FROM COLLECTIONS c
        CROSS APPLY OPENJSON(c.SEQUENTIALS, '$')
        WITH (
            order_id INT,
            post_id INT
        ) j
        WHERE c.ID IN (SELECT VALUE FROM OPENJSON(@COLLECTIONS_CHOSEN, '$'))
    )
    INSERT INTO @SEQQ(order_id, post_id)
    SELECT order_id, post_id FROM SEQ;
    --- SEQUENTIALITY POSTS
    ;
    WITH NEXTPOST AS(--- FIRST POST NOT VIEWED
        SELECT TOP 1 P.*, SEQ.order_id, SEQ.post_id
        FROM POSTS P
        LEFT JOIN @SEQQ SEQ ON SEQ.post_id = P.ID
        WHERE P.COLLECTION_ID IN (SELECT VALUE FROM OPENJSON(@COLLECTIONS_CHOSEN,
    '$'))
        AND P.ID NOT IN (
            SELECT POST_ID FROM INTERACTIONS WHERE OWNERID = @UID AND SCOPE =
    'VIEW'
        )
```

```
29          ORDER BY SEQ.order_id ASC
30      )
31      SELECT P.ID, P.AUTHOR_ID, P.TITLE, P.COMMENT, P.DC, P.DM, P.LATITUDE,
        P.LONGITUDE, P.VISIBILITY_AREA_KM, P.EXCL_DATE_START, P.EXCL_DATE_END,
        P.RECURRENT, P.COLLECTION_ID, P.VIEWERS, SEQ.order_id, SEQ.post_id, C.COLOR,
        C.ICON, C.TITLE AS CATEGORY_NAME
32      FROM POSTS P
33      LEFT JOIN @SEQQ SEQ ON SEQ.post_id = P.ID
34      INNER JOIN COLLECTIONS C ON C.ID = P.COLLECTION_ID
35      WHERE P.COLLECTION_ID IN ( SELECT VALUE FROM OPENJSON(@COLLECTIONS_CHOSEN,
        '$'))
36      AND P.ID IN ( SELECT POST_ID FROM INTERACTIONS WHERE OWNERID = @UID AND SCOPE
        = 'VIEW' )
37      ---- VIEWED POST ^
38      UNION
39          SELECT P.ID, P.AUTHOR_ID, P.TITLE, P.COMMENT, P.DC, P.DM, P.LATITUDE,
        P.LONGITUDE, P.VISIBILITY_AREA_KM, P.EXCL_DATE_START, P.EXCL_DATE_END,
        P.RECURRENT, P.COLLECTION_ID, P.VIEWERS, P.order_id, P.post_id, C.COLOR,
        C.ICON, C.TITLE AS CATEGORY_NAME
40          FROM NEXTPOST P
41      INNER JOIN COLLECTIONS C ON C.ID = P.COLLECTION_ID
42      UNION
43      --- UNION WITH POST WITHOUT SEQUENCE
44      SELECT P.ID, P.AUTHOR_ID, P.TITLE, P.COMMENT, P.DC, P.DM, P.LATITUDE,
        P.LONGITUDE, P.VISIBILITY_AREA_KM, P.EXCL_DATE_START, P.EXCL_DATE_END,
        P.RECURRENT, P.COLLECTION_ID, P.VIEWERS, NULL AS order_id, NULL AS post_id,
        C.COLOR, C.ICON, C.TITLE AS CATEGORY_NAME
45      FROM POSTS P
46      INNER JOIN COLLECTIONS C ON C.ID = P.COLLECTION_ID
47      AND (
48          --COLLECTION TIME LIMITING CHECK
49              (C.RECURRENT IS NULL and (GETDATE()> ISNULL(C.EXCL_DATE_START,
        DATEADD(DAY,-1, GETDATE()))
50                  OR GETDATE() < ISNULL(C.EXCL_DATE_END, DATEADD(DAY, 1, GETDATE())))
51              )
52          -- NO C.RECURRENTCY
53              OR  (C.RECURRENT = 1 AND (DAY(GETDATE()) BETWEEN
```

```
    DAY(C.EXCL_DATE_START) AND DAY(C.EXCL_DATE_START))) --EACH DAYS, IGNORING
    MONTHS
54              OR (C.RECURRENT = 2 AND  (DATEFROMPARTS(2000, MONTH(GETDATE()),
    DAY(GETDATE())) BETWEEN DATEFROMPARTS(2000, MONTH(C.EXCL_DATE_START),
    DAY(C.EXCL_DATE_START))
55                  AND DATEFROMPARTS(2000, MONTH(C.EXCL_DATE_END),
    DAY(C.EXCL_DATE_END)))
56          ) -- EVERY YEAR, SAME DAYS SAME MONTH
57      )
58  WHERE C.SEQUENTIALS IS NULL -- NO SEQUENTIALITY INDICATED (GENERAL  POSTS)
59      AND ( P.VIEWERS IS NULL OR P.VIEWERS = '[]' OR @UID IN (
60          SELECT VALUE FROM OPENJSON(P.VIEWERS, '$'))
61          AND
62          (
63              (P.RECURRENT IS NULL
64                  and (GETDATE()> ISNULL(P.EXCL_DATE_START, DATEADD(DAY,-1,
    GETDATE()))
65                      OR GETDATE() < ISNULL(P.EXCL_DATE_END, DATEADD(DAY, 1,
    GETDATE())))
66                  --IF NULL, MAKE IT FROM YESTERDAY TILL TOMORROW
67              )
68              -- NO P.RECURRENTCY
69                  OR
70              (P.RECURRENT = 1 AND (DAY(GETDATE()) BETWEEN
    DAY(P.EXCL_DATE_START) AND DAY(P.EXCL_DATE_START))) --EACH DAYS, IGNORING
    MONTHS
71                  OR (P.RECURRENT = 2 AND
72                  (DATEFROMPARTS(2000, MONTH(GETDATE()), DAY(GETDATE())) BETWEEN
    DATEFROMPARTS(2000, MONTH(P.EXCL_DATE_START), DAY(P.EXCL_DATE_START))
73                  AND DATEFROMPARTS(2000, MONTH(P.EXCL_DATE_END),
    DAY(P.EXCL_DATE_END)))
74              )
75              -- EVERY YEAR, SAME DAYS SAME MONTH
76          ) AND ( @UID IN (
77                  --CHECK IF THE USER REQUESTING IS ENABLED TO THAT CATEGORY
78                  SELECT VALUE FROM OPENJSON(C.VIEWERS, '$')
79              ) OR C.VIEWERS = '[]'
```

```
            )
        )
        AND C.ID IN ( SELECT VALUE FROM OPENJSON(@COLLECTIONS_CHOSEN, '$'))
        -- NB: THE VISIBILITY_AREA_KM FILTER IS MADE ON BACKEND OF THE SERVER
END;
```

**Listing 7.3:** SQL implementation for post sequentiality.

## 7.3 Docker Deployment

To deploy the Data Layer and so, implement Microsoft SQL Server, GeoMedia adopts Docker [25], which is a containerization platform that allows applications and their dependencies to be packaged and executed in isolated, reproducible environments called *containers*. This approach simplifies deployment by ensuring that the software environment required by an application remains consistent across different machines, while also providing a lightweight alternative to traditional virtual machines. Moreover, the Microsoft SQL Server image, with this techniques is independent from the underlying host operating systems and can be configured and replicate easily facilitating both the development (i.e. more instance for a production and test database) as the deployment.

In order to satisfy the GeoMedia requirements, SQL Server must be configured with a compatibility level of 130 or higher, which is required for the use of the `OPENJSON` built-in function introduced in SQL Server 2016.

To create the Docker *container*, it is sufficient to execute the terminal command shown in Listing 7.4.

```
docker run -e 'ACCEPT_EULA=Y' \ #accept lincense
    -e 'SA_PASSWORD=changeme' \ #sa password for admin
   -e 'MSSQL_PID=Express' \ #pid
   -p 1433:1433 \ #default port
   --name sqlserver \ #container name
   -d mcr.microsoft.com/mssql/server:2022-latest #the image
```

**Listing 7.4:** Docker command to create the Microsoft SQL Server container.

Subsequently, using a DBMS client, the SQL queries required to create the database, its tables, stored procedures, and other database objects are executed, resulting in the initialization of a new GeoMedia database instance.

Finally, the application layer (server http) can be configured with the connection parameters such as:

- **Address**: IPv4/IPv6 address or server name where the RDBMS is hosted. GeoMedia uses a single-server deployment, therefore `localhost` is used in the local deployment.
- **Port**: TCP/IP port used by the SQL Server instance. The default port is `1433`.
- **User**: username of a database user with the specific roles and permissions required. For GeoMedia, the user should just require permission of read and execute stored procedure, since the input and updates operations are demanded to the appropriate stored procedures. The default administrative account `sa` is avoided for security reasons.
- **Password**: password associated with the database user.
- **Database**: name of the target database to which the application connects, in this case `GEOMEDIA`.
- **Instance**: optional SQL Server instance name, whenever the server is configured with multiple independent database engine. In the GeoMedia deployment, a single SQL Server instance is used

Meanwhile the connection from the Python HTTP server to Microsoft SQL Server is provided by the library `pymssql` [56], offering an high-level API interface toward the database.

A snippet illustrating the database connection and query execution is provided in Listing 7.5, implemented through the application layer of GeoMedia.

```
# CONNECTION
def create_connection():
    return pymssql.connect(
        server=CONFIG["server"],
        user=CONFIG["user"],
        password=CONFIG["password"],
        database=CONFIG["database"],
    )
# ----------------------------------------------------
# EXECUTE QUERY
def execute_query(query, params=None, fetch=True):
```

```
    conn = create_connection()
    try:
        cursor = conn.cursor(as_dict=True)
        cursor.execute(query, params or ())
        result = []
        if fetch:
            try:
                result = cursor.fetchall()
            except Exception:
                result = []

        conn.commit()
        return result

    except Exception as e:
        conn.rollback()
        print("SQL ERROR:: ", e)
        raise

    finally:
        conn.close()
# ----------------------------------------------------
# SELECT
def select_query(query, params=None):
    return execute_query(query, params=params, fetch=True)
```

**Listing 7.5:** Python implementation of the `pymssql` database connection and query execution.

# Chapter 8

# Future works

Looking ahead, different directions for future development have been identified to enhance both functionality and user experience through the framework.

One peculiar feature is the integration of a Time Capsule system, enabling data to be stored in a small physical sensors distributed across urban environments, natural landscapes or remote places. Extending the application to integrate sensors such as Bluetooth or Near Field Communication (NFC) which working as short-range communication technologies thus to bring users physically closer to specific locations. This functionality opens the app to a vast scenarios such as gameplay, for example an interactive 'escape room' or an enhanced scavenger hunts, but also works as a memories holder, storing photographs, videos and significant content.

Altitude is a metadata currently stored during the content creation, but not yet implemented through the map. By using this dimension, GeoMedia, could support vertical placement and filtering, adaptable to high-rise buildings, mountains or drone-assisted frameworks.

As mentioned before, an improvement applicable to the architecture is the adoption of microservices-based infrastructure, thus to allow the Online Social Network to operate world-wide without suffering in terms of performance and responsivity; while promoting scalability, fault tolerance and maintainability and aligning with framework with modern cloud-native platforms.

### 8.0.1 Nostr protocol

Furthermore, future versions of GeoMedia could implement a mesh network to support a more decentralized and potentially a serverless architecture; allowing users to share content based on their physical proximity. By leveraging protocols such as Bluetooth Low Energy (BLE), or standard Bluetooth, the GeoMedia framework could establish peer-to-peer connections between nearby devices without requiring continuous access to the Internet or cellular infrastructure. A similar approach is implemented by Bitchat, a messaging application that enables users to exchange messages over Bluetooth without requiring Internet connectivity, cellular service, user accounts or centralized servers.

As Bitchat does, even GeoMedia, in addition could adopt the local peer-to-peer Nostr framework, an open and decentralized protocol that allows user to exchange information and data through distributed relays. Nostr could complement the local mesh layer by providing a synchronizing mechanism when Internet connectivity is available. In example, users could create and publish a posts related to an event, nearby devices could see it directly over the mesh network, while others users could subsequently synchronize the post. This scenario create a challenge of coordinate an hybrid architecture in which BLE communication provide local content meanwhile Nostr provides decentralized synchronization to further distance up to the maximum range imposed by GeoMedia (currently `30KM`). As result, this solution would reduce reliance ona single central server, improving the resilience of GeoMedia instance in environment where Internet connectivity is intermitted, unavailable or intentionally restricted. However, implementing this technology would introduce several challenges related to data consistency, content storage, privacy, message propagation and resource resource consumption on mobile devices.

### 8.0.2 Civil Applications and Social Impact

GeoMedia also presents an opportunity to encourage and incentivize civic participation around local events and news. As a comparison, platforms such as Waze [21] focus primarily on location-based navigation and driving assistance, demonstrating how geographically contextualized information can be used to provide users with relevant content and services as shown in Figure. GeoMedia could extend this concept beyond navigation by enabling grassroots content creation and dissemination, allowing communities to document events near them, express collective concerns or to simply share geographically contextualized knowledge.

In a democratic scenario, these capabilities may contribute to greater civic awareness, transparency and participatory governance. Instead, as opposite, in environments characterized by censorship or centralized control of information, features such as decentralized media storage and location-bound data access could potentially provide alternative channels for information dissemination and access. This aspect could make GeoMedia particularly relevant for investigating the role of location-based systems in supporting information resilience and civic communication under restrictive conditions.

However, these capabilities also raise important concerns regarding privacy, security or fake-news and misinformation. This feature would bring GeoMedia to an interdisciplinary level, examining both the civic benefits than societal risks associated with the framework, with attention to topics such as the digital freedom, censorship and ethical usage of location-based systems.

## 8.1 Feedbacks

During the SUS questionnaire, several general comments and suggestions for further development were received.

- **Content creation**: Some users suggests a faster way for creating and sharing posts, thus to foster users to publish content. One possible extension would be the adoption of QR codes, as an example, distributed across a city or in specific location that once scanned with device's camera automatically open GeoMedia application with predefined post structure (such as the collection or related metadata fields). The users would then only be required to provide a comment or to take a picture thought the app.

  The QR codes, are a type of two-dimensional matrix barcodes that could be printed in stickers or papers, in example, the Figure 8.1 represent the hyperlink for the GitHub official repository for GeoMedia framework.

- **Usability improvements**: Several usability-related suggestions were provided by the participants. One suggestion concerned the post creation phase, where media content could be expanded by default rather than requiring the user to manually activate the corresponding toggle.

  Another suggestion involved the behaviour of the bottom sheet used for limitation in post or collection creation: participants proposed enabling it to be dismissed by

tapping outside its boundaries, rather than requiring the user to interact with the dedicated closed button or by drag down the component.

The last graphical and usability suggestion is to extend the application's color theme to reflect the map theme selected, now implemented just for light-dark mode (forced thought the setting or as default, the system configuration of the device). For instance, when selecting themes such as "Retro" (light brown) or "Dark Night" (blue), the corresponding colour palette could be applied consistently across the entire app rather than being limited to the map.

- **Place research**: Participants suggested introducing the possibility of searching for specific places during the post-creation process, similarly to conventional search services that provide results for restaurants, museums or other points of interest (POIs). Although this functionality may appear in partially in contrast with GeoMedia's focus on coordinate anchored content, it would not fundamentally conflict the framework core idea, and could therefore be considered as potential future extension.

- **Profile suggestions**: Another suggestion concerned the possibility of displaying posts created by friends even out of the limited range of the posts, or to introduce a "demo mode" capable to access geographically remote content. This specific functionality could go in contrast with the content-centric nature of GeoMedia, but a less intrusive alternative could involve the prioritising or recommending posts created by users' friends.

  A related suggestion is to introduce a users ranking based on expertise or domain knowledge. For example, users identified as experts in specific field, such as history teachers, could provide more detailed and curated content. Such features, could contribute to improving the quality and reliability of content, while maintaining the location centred nature of the platform.

- **GNSS integration**: An advanced improvement could be the integration of multi-band GNSS (Global Navigation Satellite System) alongside the GPS, to improving position accuracy and provide coverage for any user on Earth including air, desert and sea zones.

- **Comments**: The final suggestion concerned the comments or instant message among users located in closer proximity to one another. This approach has been discussed previously in the context of Nostr application (subsection 8.0.1). However, the comment

feature is easily applicable and already prepared in the design of Data Layer (table `interactions`).

Another important consideration for making GeoMedia usable and engaging is ensuring that sufficient content is available across different locations. Since the platform relies on geolocated content, its usefulness is strongly influenced by the density and geographical distribution of posts. For an instance, if a first-time user opens the application in a remote or less populated area and finds little or no available content, the experience may appear empty and provide limited motivation for further interaction. Therefore, populating the platform across a broad range of locations represents an important challenge for user adoption and engagement.

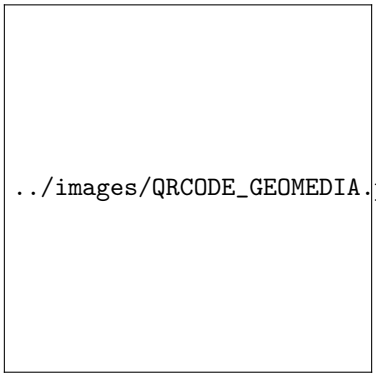


**Figure 8.1:** QR Code example, referring to GeoMedia GitHub repository.

Overall, the application received an average rating of `3.8` out of `5` from the participants.

## 8.2 Conclusion

In conclusion, GeoMedia presents an open-source, privacy-oriented, and extensible platform for location-based sharing of multimedia content, including audio, text, images, and files.

Designed as a flexible framework, the application enables users to build their own collections and define different ways to share content and interact with posts present in the environment surrounding them. This approach supports a variety of use cases while providing users with a more contextual and personalized interaction with geolocated media. Furthermore, GeoMedia integrates a range of distinctive functionalities that differentiate it from conventional social networking platforms by adopting a content-centric rather than profile-centric approach, in which posts and their spatial context constitute the core elements of the social experience.

Overall, GeoMedia demonstrates the potential of geolocated content as a means for digital connection with physical environment and social engagements. Its open an extensible and modular architecture, providing the foundations for further development and experimentation in areas such as story telling, interactive location-based games and place discovery. GeoMedia can be considered not only a platform for sharing media, but also as a geolocated media hub that explores how technology can foster meaningful interactions between people, digital content, and their surrounding environment.

# Acknowledgements

*Desidero esprimere la mia gratitudine al Prof. Claudio Enrico Palazzi, relatore della mia tesi, nonché a Dott.Lorenzo Perinello, per l'aiuto e il sostegno fornitomi durante l'intero progetto e la stesura di questa tesi.*

*Padua, September 2026* Alberto Morini

# Bibliography

## Article and papers


[1] Lorenzo Perinello. Alberto Morini Claudio Enrico Palazzi. *GoodIT 2025 - International Conference on Information Technology for Social Good.* 2025. URL: https://goodit2025.org (cit. on p. iii).

[2] Leon Ciechanowski Amin Mahmoudi Dariusz Jemielniak. *Echo Chambers in Online Social Networks: A Systematic Literature Review.* 2024. URL: https://ieeexplore.ieee.org/abstract/document/10388309 (cit. on p. 3).

[3] John Brooke. *SUS – a quick and dirty usability scale.* 1996. URL: https://www.researchgate.net/publication/319394819_SUS_--_a_quick_and_dirty_usability_scale (cit. on p. 46).

[4] Aoxia Chen and Pankaj K. Sen. *Advancement in battery technology: A state-of-the-art review.* 2016. URL: https://ieeexplore.ieee.org/document/7731812 (cit. on p. 1).

[5] Ophir Frieder. Eugene Yang David D. Lewis. *Social Network Sites: Definition, History, and Scholarship Open Access.* 2021. URL: https://arxiv.org/abs/2108.12752 (cit. on p. 3).

[6] Giulia Fioravanti et al. *Fear of missing out and social networking sites use and abuse A meta-analysis.* 2021. URL: https://www.sciencedirect.com/science/article/abs/pii/S074756322100162X (cit. on p. 6).

[7] Ombretta Gaggi. Lorenzo Perinello. *Accessibility of Mobile User Interfaces using Flutter and React Native.* 2024. URL: https://ieeexplore.ieee.org/document/10454681 (cit. on p. 45).

[8] Yuhao Zhu Matthew Halpern and Vijay Janapa Reddi. *Mobile CPU's rise to power: Quantifying the impact of generational mobile CPU design trends on performance, energy, and user satisfaction.* 2016. URL: https://ieeexplore.ieee.org/document/7446054 (cit. on p. 1).

[9] Hany Farid. Matyáš Boháček. *Protecting President Zelenskyy against Deep Fakes.* 2022. URL: https://arxiv.org/abs/2206.12043 (cit. on p. 3).

[10] Zahra Khanbabaloo. Mohammad Hajarian. *Toward Stopping Incel Rebellion: Detecting Incels in Social Media Using Sentiment Analysis.* 2021. URL: https://www.researchgate.net/profile/Mohammad-Hajarian/publication/352100924_Toward_Stopping_Incel_Rebellion_Detecting_Incels_in_Social_Media_Using_Sentiment_Analysis/links/60b8e34992851cb13d73e858/Toward-Stopping-Incel-Rebellion-Detecting-Incels-in-Social-Media-Using-Sentiment-Analysis.pdf.

[11] Andrew Tomk. Ravi Kum Jasmine Novak. *Structure and Evolution of Online Social Networks.* 2006. URL: https://dl.acm.org/doi/abs/10.1145/1150402.1150476 (cit. on p. 2).

[12] Dmitri Rozgonjuk et al. *Fear of Missing Out (FoMO) and social media's impact on daily life and productivity at work: Do WhatsApp, Facebook, Instagram, and Snapchat use disorders mediate that association?* 2020. URL: https://www.sciencedirect.com/science/article/abs/pii/S0306460320306171 (cit. on p. 6).

[13] Roger W. Sinnott. *Virtues of the Haversine.* 1984 (cit. on p. 19).

[14] T. Vincenty. *Direct and inverse solutions of geodesics on the ellipsoid with application of nested equations.* 1975. URL: https://www.tandfonline.com/doi/abs/10.1179/sre.1975.23.176.88 (cit. on p. 19).

## Web sites

[15] Anitha Edison Amala Mary. *Deep fake Detection using deep learning techniques A Literature Review.* 2023. URL: https://ieeexplore.ieee.org/abstract/document/10164881/authors#authors (cit. on p. 3).

[16] Shariq Aziz Butt et al. *Remote mobile health monitoring frameworks and mobile applications: Taxonomy, open challenges, motivation, and recommendations.* 2024. URL: https:

//www.sciencedirect.com/science/article/abs/pii/S0952197624003919 (cit. on p. 1).

[17] Windows Central. *Windows Phone isn't dead, Part VI: App Gap? Microsoft has a platform for that.* 2016. URL: https://www.windowscentral.com/windows-phone-isnt-dead-part-vi-app-gap (cit. on p. 2).

[18] Tech Crunch. *Instagram rolls out a new searchable map to make it easier to discover popular locations.* 2022. URL: https://techcrunch.com/2022/07/19/instagram-new-searchable-map-experience/ (cit. on p. 8).

[19] Demandsage. *Android usage in 2026.* 2026. URL: https://www.demandsage.com/android-statistics/ (cit. on p. 48).

[20] Google. *Google Maps pricing up to 2026.* 2026. URL: https://developers.google.cn/maps/billing-and-pricing/pricing?hl=en (cit. on p. 35).

[21] Waze Mobile Ltd. Google. *Waze official page.* 2006. URL: https://www.waze.com/apps (cit. on pp. 2, 83).

[22] Jeffrey Gottfried. *Americans' Social Media Use: YouTube and Facebook are by far the most used online platforms among U.S. adults; TikTok's user base has grown since 2021.* 2024. URL: https://www.pewresearch.org/internet/2024/01/31/americans-social-media-use/ (cit. on p. 2).

[23] Apple Inc. *iOS official page.* 2007. URL: https://www.apple.com/os/ios/ (cit. on p. 2).

[24] Augglin Inc. *Auglinn official page.* 2024. URL: https://www.auglinn.com/ (cit. on p. 11).

[25] Docker Inc. *Docker official page.* 2026. URL: https://www.docker.com/ (cit. on p. 79).

[26] Google Inc. *Android Auto official page.* 2015. URL: https://www.android.com/auto/ (cit. on p. 10).

[27] Google Inc. *Android official page.* 2008. URL: https://www.android.com/ (cit. on p. 2).

[28] Google Inc. *Google Lens official page.* 2017. URL: https://lens.google/ (cit. on p. 2).

[29] Google Inc. *Google Maps official page*. 2005. URL: https://www.google.com/maps/about/ (cit. on pp. 2, 10).

[30] Google Inc. *Google shuts failed social network Google*. 2019. URL: https://www.bbc.com/news/technology-47771927 (cit. on p. 5).

[31] Google Inc. *Google Street View*. 2007. URL: https://www.google.com/streetview/ (cit. on p. 10).

[32] Google Inc. *See where your friends are with Google Latitude*. 2009. URL: https://googleblog.blogspot.com/2009/02/see-where-your-friends-are-with-google.html (cit. on p. 5).

[33] Jagat Inc. *Jagat official page*. 2023. URL: https://www.jagat.io/ (cit. on p. 11).

[34] Meta Inc. *Meta Reports Fourth Quarter and Full Year 2023 Results; Initiates Quarterly Dividend*. 2024. URL: https://investor.atmeta.com/investor-news/press-release-details/2024/Meta-Reports-Fourth-Quarter-and-Full-Year-2023-Results-Initiates-Quarterly-Dividend/default.aspx (cit. on p. 2).

[35] Meta Inc. *Scaling the Instagram Explore recommendations system*. 2023. URL: https://engineering.fb.com/2023/08/09/ml-applications/scaling-instagram-explore-recommendations-system/ (cit. on p. 8).

[36] Snap Inc. *Snap Inc. Announces Third Quarter 2025 Financial Results*. 2025. URL: https://investor.snap.com/news/news-details/2025/Snap-Inc--Announces-Third-Quarter-2025-Financial-Results/default.aspx (cit. on p. 6).

[37] Snap Inc. *Snapchat official page*. 2011. URL: https://www.snapchat.com/ (cit. on p. 6).

[38] Marina Niessner. J. Anthony Cookson William Mullins. *Social Media and Finance*. 2024. URL: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=4806692 (cit. on p. 2).

[39] Steven Furnell. Karl van der Schyff Stephen Flowerday. *Duplicitous social media and data surveillance An evaluation of privacy risk*. 2020. URL: https://www.sciencedirect.com/science/article/abs/pii/S0167404820300961 (cit. on p. 3).

[40] Andrei O. J. Kwok and Sharon G. M. Koh. *Deepfake: a social construction of technology perspective*. 2020. URL: https://www.tandfonline.com/doi/abs/10.1080/13683500.2020.1738357 (cit. on p. 3).

[41] Peter James Marcin Straczkiewicz and Jukka-Pekka Onnela. *A systematic review of smartphone-based human activity recognition methods for health research.* 2021. URL: https://www.nature.com/articles/s41746-021-00514-4 (cit. on p. 1).

[42] ABC News. *Snapchat's new Snap Map feature raises privacy concerns.* 2017. URL: https://abcnews.com/Lifestyle/snapchats-snap-map-feature-raises-privacy-concerns/story?id=48271889 (cit. on p. 6).

[43] Neliia Blynova Oksana Kyrylova and Viktoriia Pavlenko. *Mobile internet usage worldwide - statistics and facts.* 2023. URL: https://www.statista.com/topics/779/mobile-internet/.

[44] Neliia Blynova Oksana Kyrylova and Viktoriia Pavlenko. *The perspectives for mobile application use in media education.* 2023. URL: https://www.tandfonline.com/doi/full/10.1080/10494820.2023.2186897 (cit. on p. 1).

[45] S.A. OutSystems- Software em Rede. *IonicFramework official page.* 2013. URL: https://ionicframework.com/ (cit. on p. 32).

[46] A. Perrin. *Social Media Usage: 2005–2015 – 65% of adults now use social networking sites, a nearly tenfold jump in the past decade.* 2015. URL: https://www.secretintelligenceservice.org/wp-content/uploads/2016/02/PI_2015-10-08_Social-Networking-Usage-2005-2015_FINAL.pdf (cit. on p. 2).

[47] Amanda Raffoul et al. *Social media platforms generate billions of dollars in revenue from U.S. youth Findings from a simulated revenue model.* 2023. URL: https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0295337 (cit. on p. 2).

[48] Reuters. *Meta CEO Zuckerberg says Instagram has grown to 3 billion monthly active users.* 2025. URL: https://www.reuters.com/business/meta-ceo-zuckerberg-says-instagram-has-grown-3-billion-monthly-active-users-2025-09-24/ (cit. on p. 8).

[49] Apple Inc. Shazam Entertainment Limited. *Shazam official page.* 2002. URL: https://www.shazam.com/ (cit. on p. 2).

[50] Meta Open Source. *React Native directory for external packages.* 2015. URL: https://reactnative.directory/ (cit. on p. 33).

[51] Meta Open Source. *React Native official page*. 2015. URL: https://reactnative.dev/ (cit. on p. 31).

[52] Statista. *Device usage of Facebook users worldwide as of January 2022*. 2022. URL: https://www.statista.com/statistics/377808/distribution-of-facebook-users-by-device/ (cit. on p. 1).

[53] Statista. *Global smartphone sales to end users from 2007 to 2023*. 2023. URL: https://www.statista.com/statistics/263437/global-smartphone-sales-to-end-users-since-2007/ (cit. on p. 1).

[54] Statista. *M Number of monthly active Instagram users from January 2013 to December 2021* . 2021. URL: https://www.statista.com/statistics/253577/number-of-monthly-active-instagram-users/ (cit. on p. 8).

[55] Statista. *Most popular global mobile messenger apps as of October 2025, based on number of monthly active users*. 2025. URL: https://www.statista.com/statistics/258749/most-popular-global-mobile-messenger-apps/ (cit. on p. 1).

[56] Mikhail Terekhov. *PyMSSQL official page, Python library for Microsoft SQL Server*. 2014. URL: https://pypi.org/project/pymssql/ (cit. on p. 80).

[57] Social Media Today. *Instagram Stories is Now Being Used by 500 Million People Daily*. 2019. URL: https://www.socialmediatoday.com/news/instagram-stories-is-now-being-used-by-500-million-people-daily/547270/ (cit. on p. 8).

[58] Eastern District of New York. U.S. Attorney's Office. *Two Individuals Arrested for Publishing AI Deepfake Pornography In Violation Of TAKE IT DOWN Act*. 2026. URL: https://www.justice.gov/usao-edny/pr/two-individuals-arrested-publishing-ai-deepfake-pornography-violation-take-it-down-act (cit. on p. 3).

[59] International Telecommunication Union. *International Telecommunication Union. 2023. Facts and Figures 2023 – Mobile Phone Ownership*. 2023. URL: https://www.itu.int/itu-d/reports/statistics/2023/10/10/ff23-mobile-phone-ownership/ (cit. on p. 1).

[60] Computer World. *Windows Phone still has serious app gap despite Microsoft's $7.2 billion Nokia buyout*. 2013. URL: https://www.computerworld.com/article/

1500308/windows-phone-still-has-serious-app-gap-despite-microsoft-s-7-2-billion-nokia-buyo.html (cit. on p. 2).